

**LAPORAN AKHIR PROJECT MACHINE LEARNING KLASIFIKASI RISIKO
GAGAL BAYAR NASABAH MENGGUNAKAN ALGORITMA RANDOM FOREST
DENGAN PENDEKATAN PENANGANAN IMBALANCED DATA (SMOTE)
UNTUK MENDUKUNG KEBIJAKAN MANAJEMEN RISIKO KREDIT**

UJIAN AKHIR SEMESTER MATA KULIAH
PEMBELAJARAN MESIN



Oleh :

DANDY PRASETYO NUGROHO

A11.2024.15645

**PROGRAM STUDI SARJANA TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS DIAN NUSWANTORO SEMARANG**

2026

PETA PEMENUHAN SOAL UJIAN AKHIR SEMESTER

Tabel di bawah ini merangkum pemetaan jawaban soal UAS mata kuliah Pembelajaran Mesin dengan bobot penilaiannya masing-masing. Total bobot untuk keempat soal adalah 85%, sedangkan sisanya dialokasikan untuk aspek dokumentasi dan kualitas penyajian laporan.

Soal	Bobot	Topik	Lokasi pada Laporan
Soal 1	10%	Problem Definition & Data Acquisition	BAB I Pendahuluan
Soal 2	20%	Exploratory Data Analysis & Preprocessing	BAB III Analisis Dataset & Representasi Data, BAB IV Perancangan Pipeline
Soal 3	30%	Modeling & Evaluation	BAB II Perumusan Masalah (2.4), BAB V Strategi Pemodelan & Eksperimen
Soal 4	25%	Deployment & Streamlit Application	BAB VI Rencana Deployment Model, BAB VII Deployment Aplikasi Streamlit
Soal 5	15%	Documentation & Presentation	Terpisah dengan Laporan

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Kredit adalah pilar utama dalam operasional perbankan. Namun, setiap pengajuan pinjaman selalu diiringi tantangan bagi lembaga keuangan untuk memprediksi apakah calon nasabah akan melunasi kewajibannya atau justru mengalami gagal bayar (*default*). Selama ini, penilaian tersebut umumnya dilakukan secara manual oleh analis kredit dengan meninjau profil finansial seperti pekerjaan, pendapatan, dan tabungan nasabah.

Tantangan terbesarnya adalah kasus gagal bayar relatif jarang terjadi. Berdasarkan dataset yang kami gunakan, hanya ada 333 dari 10.000 nasabah (3,33%) yang tergolong *default*, sementara sisanya adalah nasabah lancar. Rasio ketimpangan data yang mencapai 29:1 ini membuat pola risiko gagal bayar sangat sulit dideteksi melalui aturan manual atau logika sederhana.

Kesalahan dalam penilaian risiko juga membawa risiko finansial bagi bank. Jika nasabah berisiko tinggi lolos (*false negative*), bank akan merugi akibat kredit macet. Sebaliknya, jika nasabah yang layak justru ditolak (*false positive*), bank kehilangan potensi keuntungan. Oleh karena itu, kita memerlukan model klasifikasi yang tidak hanya akurat, tetapi juga mampu menangani masalah data yang tidak seimbang (*imbalanced data*).

Dalam proyek ini, kami membangun model klasifikasi menggunakan algoritma *Random Forest* yang dikombinasikan dengan teknik *SMOTE* (*Synthetic Minority Over-sampling Technique*) agar model tidak bias ke kelas mayoritas. Selain itu, kami juga membandingkan performa model ini dengan *Logistic Regression* dan *XGBoost*. Harapannya, hasil klasifikasi ini dapat membantu tim manajemen risiko dalam memproses pengajuan pinjaman secara lebih objektif.

1.2 Tujuan Bisnis dan Analisis

Dari sisi praktis, keberadaan model ini dirancang sebagai pendukung keputusan bagi divisi manajemen risiko kredit. Model ini berfungsi sebagai sistem deteksi dini (*early warning system*) yang mampu melakukan *screening* awal terhadap setiap pengajuan pinjaman baru sebelum data tersebut ditinjau lebih lanjut oleh analis kredit secara manusia. Dengan adanya sistem ini, proses evaluasi diharapkan bisa berjalan lebih efisien tanpa menghilangkan peran krusial *human judgment* dalam pengambilan keputusan akhir.

Sementara itu, dari sudut pandang analisis data, fokus utama proyek ini tidak sekadar mengejar tingkat akurasi yang tinggi secara keseluruhan (*overall accuracy*). Kami memberikan perhatian khusus pada sensitivitas model dalam mengidentifikasi kelas minoritas, yaitu nasabah yang berpotensi *default*.

Mengapa ini penting? Karena dalam kasus ketimpangan data yang ekstrem, model standar cenderung mengabaikan kelas minoritas demi memaksimalkan akurasi pada kelas mayoritas. Padahal, justru kelompok nasabah *default* inilah yang paling bernilai untuk dideteksi oleh bank guna meminimalisir risiko kerugian. Oleh karena itu, melalui penerapan teknik *oversampling* dan penggunaan algoritma berbasis *ensemble*, kami bertujuan menciptakan model yang lebih adil, peka terhadap risiko, dan tidak terjebak pada bias kelas mayoritas yang dominan dalam dataset.

1.3 Metrik Kesuksesan Proyek

Karena fokus utama kita adalah mendeteksi kelas minoritas, metrik akurasi saja jelas tidak cukup. Untuk menilai apakah model ini benar-benar berhasil, kita menggunakan beberapa metrik berikut:

- **Recall untuk kelas *default*:** Ini krusial, karena kita ingin meminimalisir *false negative*. Gagal mendeteksi nasabah berisiko jauh lebih merugikan secara finansial bagi bank daripada sebaliknya.
- **PR-AUC (*Precision-Recall AUC*):** Kita memilih ini sebagai metrik utama—bukan ROC-AUC—karena jauh lebih representatif untuk menangani kasus data yang tidak seimbang (*imbalanced data*) secara ekstrem.
- **F1-Score untuk kelas *default*:** Digunakan sebagai jalan tengah antara *recall* dan *precision*. Tujuannya supaya model tidak asal menebak positif hanya demi mengejar *recall* yang tinggi.

Target minimal yang kita tetapkan sejak awal adalah *recall* di atas 0,70 dengan *precision* antara 0,30 sampai 0,40. Berdasarkan hasil eksperimen di BAB V, target *recall* berhasil tercapai (0,7400 pada model *Random Forest*). Memang, *precision* aktual kita berada di angka 0,2056, sedikit di bawah target awal. Tapi, ini adalah *trade-off* yang wajar mengingat rasio ketimpangan data kita yang cukup berat (29:1). Analisis lebih mendalam mengenai hal ini akan kita bahas di BAB V.5.10.

1.4 Sumber dan Identitas Dataset

Untuk proyek ini, kita menggunakan *dataset* lokal bernama `Default_Fin.csv`. Isinya adalah data finansial nasabah, seperti status pekerjaan, saldo bank, pendapatan tahunan, dan status *default*. Dataset ini berbentuk numerik terstruktur yang memang sudah disiapkan untuk keperluan akademis, sehingga polanya sangat umum ditemui dalam studi kasus *credit scoring*. Berikut adalah ringkasan profil data yang kita gunakan:

Atribut	Detail
Nama Dataset	Default_Fin.csv (Loan/Credit Default Prediction Dataset)
Sumber	Berkas CSV lokal — data finansial nasabah
Ukuran	10.000 baris nasabah, 5 kolom atribut (termasuk Index dan label)
Label	Defaulted? (0 = tidak default, 1 = default)
Status	Dataset numerik terstruktur, tidak memiliki missing values maupun baris duplikat

1.5 Statistik Deskriptif Awal

Sebelum masuk ke tahap pembersihan data, kita perlu memahami profil *dataset* mentah yang digunakan. Secara keseluruhan, data ini terdiri dari 10.000 baris dan 5 kolom, yang mencakup kolom *Index* (identitas nasabah), *Employed* (biner), dua fitur numerik kontinu (*Bank Balance* dan *Annual Salary*), serta satu target biner (*Defaulted?*). Hasil pengecekan menggunakan fungsi `isnull().sum()` dan `duplicated().sum()` di

notebook eksperimen mengonfirmasi bahwa data ini sudah cukup bersih; tidak ada *missing values* maupun baris duplikat yang perlu ditangani. Berikut adalah ringkasan statistik deskriptif dari data tersebut:

Atribut	Detail
Jumlah Baris	10.000 baris data nasabah
Jumlah Kolom	5 kolom (Index + 3 fitur input + 1 target)
Nilai Hilang	0 (tidak ditemukan missing values pada seluruh kolom)
Baris Duplikat	0 (tidak ditemukan baris duplikat)
Distribusi Target	9.667 baris (96,67%) kategori tidak default, 333 baris (3,33%) kategori default, rasio imbalance 29:1
Employed (mean)	0,7056 — sekitar 70,56% nasabah berstatus bekerja
Bank Balance	rentang 0 — 31.851,84; rata-rata 10.024,50; std 5.804,58
Annual Salary	rentang 9.263,64 — 882.650,76; rata-rata 402.203,78; std 160.039,68

Berdasarkan ringkasan statistik di atas, terlihat jelas bahwa fitur *Bank Balance* dan *Annual Salary* memiliki rentang nilai yang cukup lebar dengan standar deviasi yang tinggi. Hal ini mengonfirmasi perlunya tahapan *feature scaling* (seperti normalisasi atau standarisasi) pada langkah pemrosesan data selanjutnya agar model tidak bias terhadap fitur dengan rentang nilai yang lebih besar. Selain itu, proporsi nasabah yang bekerja (*Employed*) sebesar 70,56% menjadi salah satu variabel yang akan kita perhatikan pengaruhnya terhadap probabilitas *default* nasabah.

BAB II PERUMUSAN MASALAH

2.1 Identifikasi Masalah — Mengapa Machine Learning Diperlukan

Selama ini, penilaian kredit seringkali masih mengandalkan pendekatan konvensional berbasis aturan kaku (*rule-based*), misalnya dengan langsung menolak pengajuan jika saldo bank nasabah di bawah angka tertentu. Meskipun terlihat praktis, metode manual seperti ini memiliki keterbatasan mendasar dalam menilai risiko kredit secara objektif:

- **Gagal menangkap interaksi antar variabel:** Aturan statis tidak mampu melihat kaitan kompleks antara pekerjaan, saldo, dan pendapatan secara bersamaan. Padahal, risiko gagal bayar biasanya merupakan akumulasi dari berbagai faktor yang saling memengaruhi, bukan sekadar satu kondisi tunggal.
- **Subjektivitas ambang batas (*threshold*):** Menentukan *threshold* secara manual sangat subjektif. Angka yang dianggap "aman" oleh analis belum tentu merupakan titik pemisah yang optimal jika dilihat secara statistik dari data historis.
- **Tidak adaptif:** Sistem *rule-based* tidak bisa belajar dari data masa lalu, sehingga ia tidak mampu berkembang atau memperbaiki akurasinya sendiri seiring bertambahnya data nasabah yang baru.

Di sinilah *Machine Learning*, khususnya model *Random Forest*, hadir sebagai solusi. Sebagai model *ensemble* yang menggabungkan banyak *decision tree*, ia mampu mempelajari pola keputusan non-linear dan interaksi antar fitur secara otomatis langsung dari data. Selain lebih tangguh menghadapi *outlier*, *Random Forest* juga menyediakan *feature importance*. Fitur ini memberikan wawasan berharga bagi manajemen risiko untuk memahami faktor apa yang paling dominan dalam memicu gagal bayar—sebuah wawasan yang hampir mustahil didapat dari aturan manual yang kaku.

2.2 Rumusan Masalah

Berdasarkan identifikasi masalah di atas, fokus utama yang akan dibahas dalam proyek ini adalah:

- Bagaimana cara membangun model klasifikasi *Random Forest* yang efektif untuk memprediksi risiko gagal bayar (*default*) nasabah berdasarkan profil finansial mereka?
- Sejauh mana ketimpangan kelas (rasio 29:1) memengaruhi performa model, dan teknik penanganan *imbalanced data* mana (seperti *class weighting* atau *SMOTE*) yang paling efektif untuk kasus ini?
- Faktor-faktor apa saja yang paling krusial dalam menentukan risiko gagal bayar nasabah jika ditinjau melalui *feature importance* dan interpretasi SHAP?
- Seberapa andal model yang kita bangun dalam mengidentifikasi nasabah berisiko tinggi jika dievaluasi menggunakan metrik yang tepat (*Precision*, *Recall*, *F1-Score*, hingga *PR-AUC*) dibandingkan hanya mengandalkan metrik akurasi biasa?

2.3 Tujuan Proyek

Secara keseluruhan, proyek ini bertujuan untuk:

- **Membangun *pipeline machine learning* yang menyeluruh**
Kita akan melakukan alur kerja *end-to-end*, mulai dari pembersihan data (*data cleaning*), rekayasa fitur (*feature engineering*), penanganan data yang tidak seimbang (*imbalance*), hingga melatih dan membandingkan performa tiga model klasifikasi, yakni *Random Forest*, *Logistic Regression*, dan *XGBoost*.
- **Mencari skenario terbaik untuk data tidak seimbang**
Melalui eksperimen terukur menggunakan *Stratified 5-Fold Cross-Validation*, kita akan menguji berbagai metode penanganan *imbalanced data* untuk menemukan mana yang paling efektif dalam meningkatkan sensitivitas model terhadap kelas minoritas.
- **Implementasi model ke dalam alat bantu Keputusan**
Hasil akhir dari proyek ini adalah model terbaik yang sudah melalui proses *tuning* dan siap digunakan sebagai *prototype* aplikasi berbasis *Streamlit*, yang nantinya dapat membantu *credit analyst* dalam menyaring pengajuan pinjaman secara lebih objektif.

2.4 Definisi Learning Task

Proyek ini dikategorikan ke dalam **Supervised Learning — Klasifikasi Biner (*Binary Classification*)**. Alasannya sederhana:

- **Data historis sudah berlabel**
Kita memiliki kolom *Defaulted?* yang berfungsi sebagai label dengan nilai biner (0 untuk nasabah lancar dan 1 untuk *default*).
- **Proses pelatihan yang terarah**
Model dilatih dengan mencocokkan data input (kondisi finansial nasabah) dengan output yang sudah diketahui (status *default*), sehingga model bisa "belajar" pola dari data tersebut.
- **Tujuan yang jelas**
Fokus kita adalah mengategorikan nasabah ke dalam dua kelas yang terpisah (klasifikasi), bukan memprediksi angka kontinu (regresi).

Untuk mempermudah pemahaman, berikut adalah variabel yang kita gunakan:

- **Target/Label Variable:** *Defaulted?* (variabel biner; 0 = lancar, 1 = gagal bayar).
- **Input Features (sebelum *feature engineering*):**
 - *Employed* (status pekerjaan: 1 jika bekerja, 0 jika tidak).
 - *Bank Balance* (saldo tabungan nasabah).
 - *Annual Salary* (pendapatan tahunan nasabah).

2.5 Hipotesis Awal

Secara teoretis, kami menduga adanya korelasi yang cukup kuat antara variabel *Bank Balance* dan *Annual Salary* dengan risiko gagal bayar. Logikanya, perbandingan antara besaran simpanan dan pendapatan mencerminkan *financial cushion* (bantalan finansial) nasabah; semakin rendah bantalan ini saat menghadapi kewajiban cicilan, maka semakin tinggi pula probabilitas nasabah tersebut akan mengalami *default*. Oleh karena itu, kami memprediksi bahwa model *Random Forest*, yang didukung dengan teknik *oversampling* seperti *SMOTE*, akan mampu menangkap pola non-linear ini secara jauh lebih akurat dibandingkan metode *rule-based* konvensional.

Kami menargetkan model mampu mencapai nilai *Recall* di atas 0,70 untuk kelas minoritas. Target ini kami tetapkan sebagai komitmen untuk meminimalkan *false negative*, mengingat dampak kerugian finansial yang jauh lebih besar bagi lembaga keuangan jika nasabah berisiko tinggi lolos dari deteksi. Meski demikian, kami memprediksi nilai *Precision* akan berada pada kisaran 0,30–0,40. Angka yang terlihat moderat ini kami anggap sebagai konsekuensi logis dari rasio ketimpangan data yang sangat tinggi (29:1), di mana model secara inheren akan cenderung lebih "agresif" dalam mengklasifikasikan nasabah sebagai *default* demi mengejar *recall*.

Sebagai catatan kejujuran akademik, hipotesis ini disusun sepenuhnya sebelum eksperimen dijalankan. Setelah melalui pengujian aktual yang dirinci pada BAB V, target *Recall* berhasil dilampaui dengan capaian 0,7400 pada model *Random Forest* final. Sementara itu, *Precision* aktual berada di angka 0,2056, sedikit di bawah kisaran target 0,30–0,40. Hasil ini kami laporkan apa adanya karena mencerminkan *trade-off* nyata yang tidak terelakkan saat menangani data dengan ketimpangan ekstrem. Kondisi ini sejalan dengan temuan He dan Garcia (2009) yang menekankan adanya batasan inheren pada performa *precision* ketika model dihadapkan pada kelas minoritas dengan populasi yang sangat kecil dalam dataset besar.

BAB III ANALISIS DATASET & REPRESENTASI DATA

3.1 Sumber dan Identitas Dataset

Sebagaimana telah kita ulas pada BAB I.4, sumber utama data dalam proyek ini adalah *dataset* Default_Fin.csv. *Dataset* ini berisi 10.000 entri data nasabah yang dikemas dalam 5 kolom atribut finansial. Seluruh proses pemuatan data dilakukan secara terprogram menggunakan fungsi `pd.read_csv()` di dalam *notebook* "Eksperimen_Klasifikasi_Risiko_Gagal_Bayar.ipynb". Berdasarkan pengecekan teknis menggunakan atribut `.shape`, dimensi data dikonfirmasi tepat berjumlah (10.000, 5), yang memastikan integritas data tetap terjaga sebelum melangkah ke tahapan analisis berikutnya.

3.2 Struktur Data dan Tipe Fitur

Secara arsitektur, *dataset* ini merupakan data terstruktur (*structured data*) dalam format tabular CSV. Setiap baris di dalamnya secara unik merepresentasikan satu profil nasabah beserta status kewajiban pembayarannya. Berbeda dengan *dataset* transaksi ritel yang biasanya kaya akan atribut kategorikal, *dataset* ini lebih bersifat ringkas dan murni finansial. Oleh karena itu, tantangan utama yang kita hadapi dalam proyek ini bukanlah kompleksitas jumlah fitur, melainkan pada ketimpangan distribusi kelas target yang sangat tajam antara nasabah yang *default* dan yang tidak. Secara teknis, fitur-fitur yang tersedia dalam *dataset* ini diklasifikasikan ke dalam tiga kategori utama:

- **Numerik Kontinu (*float*):** Mencakup fitur *Bank Balance* dan *Annual Salary* yang merepresentasikan nilai moneter.
- **Numerik Biner/Diskrit (*integer*):** Mencakup fitur *Employed* dan *Defaulted?* (label target).
- **Identitas (*Identifier*):** Kolom *Index* yang berfungsi sebagai penanda urut nasabah. Karena kolom ini tidak memiliki nilai prediktif terhadap perilaku gagal bayar, kita akan melakukan *drop* pada tahap *cleaning* agar tidak memengaruhi performa model.

3.3 Tabel Deskripsi Fitur

Untuk memberikan gambaran yang lebih komprehensif mengenai karakteristik setiap variabel, berikut adalah ringkasan teknis dari fitur yang kita gunakan:

No	Nama Fitur	Tipe Data	Peran	Deskripsi
1	Index	Integer	Drop	Nomor urut nasabah, tidak memiliki nilai prediktif
2	Employed	Biner (0/1)	Input	Status pekerjaan nasabah; 70,56% nasabah berstatus bekerja
3	Bank Balance	Float	Input	Saldo tabungan nasabah (rentang 0 — 31.851,84; rata-rata 10.024,50)
4	Annual Salary	Float	Input	Pendapatan tahunan nasabah (rentang 9.263,64 — 882.650,76; rata-rata 402.203,78)

No	Nama Fitur	Tipe Data	Peran	Deskripsi
5	Defaulted?	Biner (0/1)	TARGET (y)	9.667 nasabah tidak default (96,67%), 333 nasabah default (3,33%)

3.4 Analisis Kualitas Data

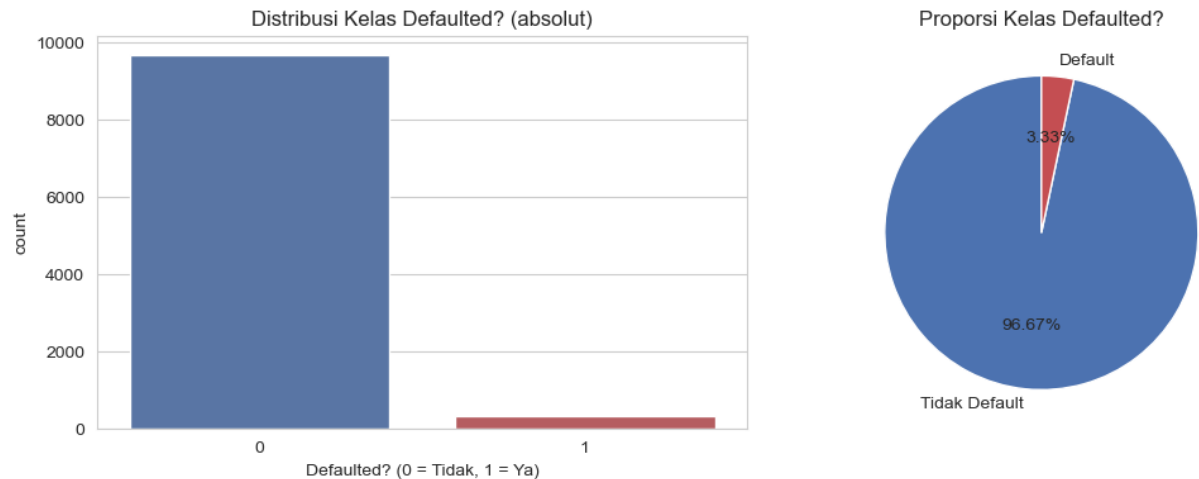
Langkah krusial sebelum membangun model adalah memastikan bahwa data yang digunakan memiliki integritas tinggi. Dalam *notebook* eksperimen, kami melakukan pengecekan kualitas data secara eksplisit menggunakan fungsi `isnull().sum()` untuk mendeteksi *missing values* dan `duplicated().sum()` untuk mengidentifikasi redundansi data. Selain itu, kami menerapkan metode *Interquartile Range* (IQR) untuk mendeteksi potensi *outlier* pada variabel finansial. Hasil audit kualitas data tersebut dirangkum dalam tabel berikut:

Aspek	Hasil Pengecekan
Missing Values	0 nilai hilang pada seluruh kolom (Index, Employed, Bank Balance, Annual Salary, Defaulted?)
Baris Duplikat	0 baris duplikat ditemukan (<code>duplicated().sum()</code> = 0)
Outlier Bank Balance (IQR)	Batas [-6.541,51 ; 26.317,96] — 31 baris (0,31%) terdeteksi sebagai outlier
Outlier Annual Salary (IQR)	Batas [-148.325,34 ; 930.103,62] — 0 baris (0,00%) terdeteksi sebagai outlier
Keputusan Outlier	Outlier pada Bank Balance tidak dibuang secara otomatis, karena nasabah bersaldo sangat tinggi maupun sangat rendah bisa jadi merupakan sinyal risiko yang relevan secara bisnis, bukan noise pengukuran

Berdasarkan hasil di atas, dapat disimpulkan bahwa dataset dalam kondisi yang sangat bersih—tanpa ada nilai yang hilang maupun entri duplikat yang berpotensi merusak bobot model. Terkait temuan 31 baris *outlier* pada fitur *Bank Balance*, kami memutuskan untuk tidak melakukan penghapusan data. Dalam konteks penilaian risiko kredit, nasabah dengan saldo yang sangat tinggi maupun sangat rendah sering kali merupakan entitas yang paling kritis untuk diamati. Dengan demikian, data tersebut tetap dipertahankan guna menjaga realitas profil nasabah di dalam model, karena nilai ekstrem tersebut merupakan informasi bisnis yang sah dan bukan kesalahan teknis dalam input data.

3.5 Analisis Univariat dan Multivariat

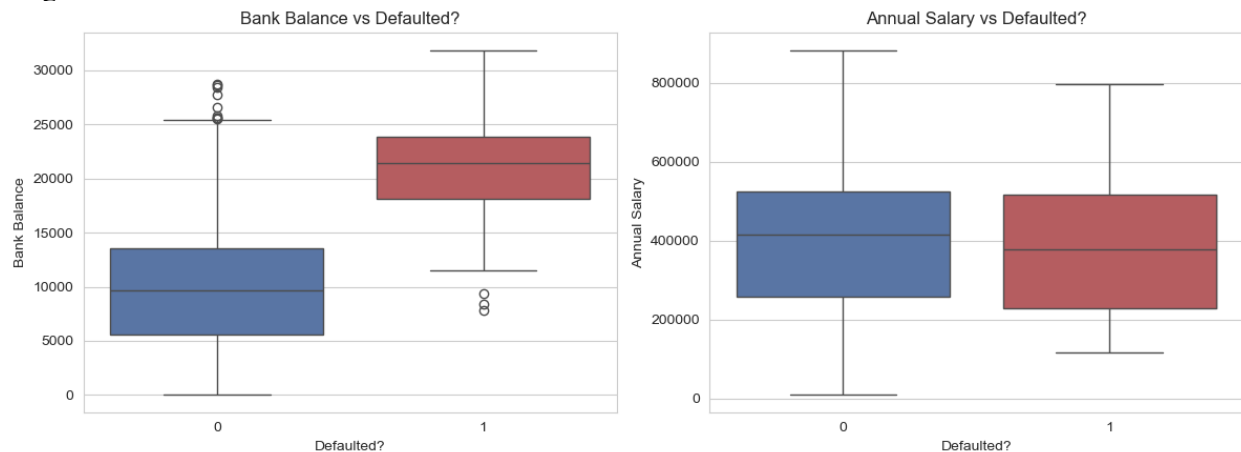
Langkah pertama dalam memahami karakteristik data adalah melihat distribusi kelas target Defaulted?. Visualisasi dalam bentuk grafik batang dan *pie chart* (Gambar 3.1) mempertegas fakta bahwa kita sedang berhadapan dengan ketimpangan kelas yang cukup ekstrem, yakni dengan rasio 29:1.



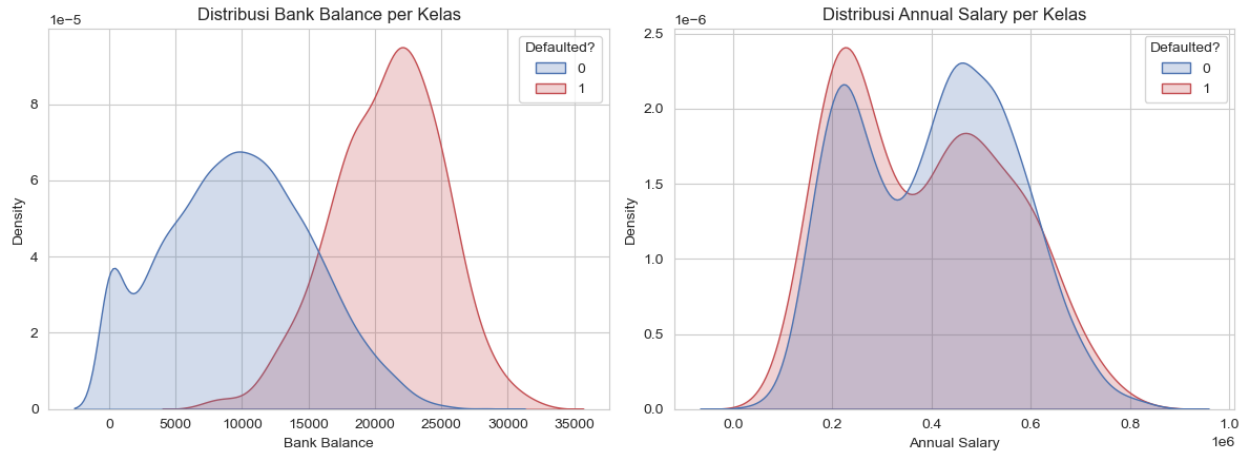
Gambar 3.1 Distribusi Kelas Defaulted? (Absolut dan Proporsi)

Rasio ini merupakan inti dari permasalahan dalam penelitian ini. Jika model klasifikasi standar dilatih tanpa penanganan khusus pada data ini, ia akan cenderung mengalami *bias* ke arah kelas mayoritas. Model mungkin akan mencatatkan akurasi secara semu hingga 96,67% hanya dengan "menebak" semua nasabah sebagai *non-default*. Namun, dalam konteks manajemen risiko kredit, model seperti ini sama sekali tidak berguna karena ia gagal menjalankan fungsi utamanya: mendeteksi nasabah berisiko tinggi.

Lebih jauh lagi, kami melakukan analisis multivariat untuk melihat bagaimana fitur finansial memengaruhi status *default*. Gambar 3.2 (*boxplot*) dan Gambar 3.3 (*KDE plot*) menunjukkan pola yang sangat menarik:



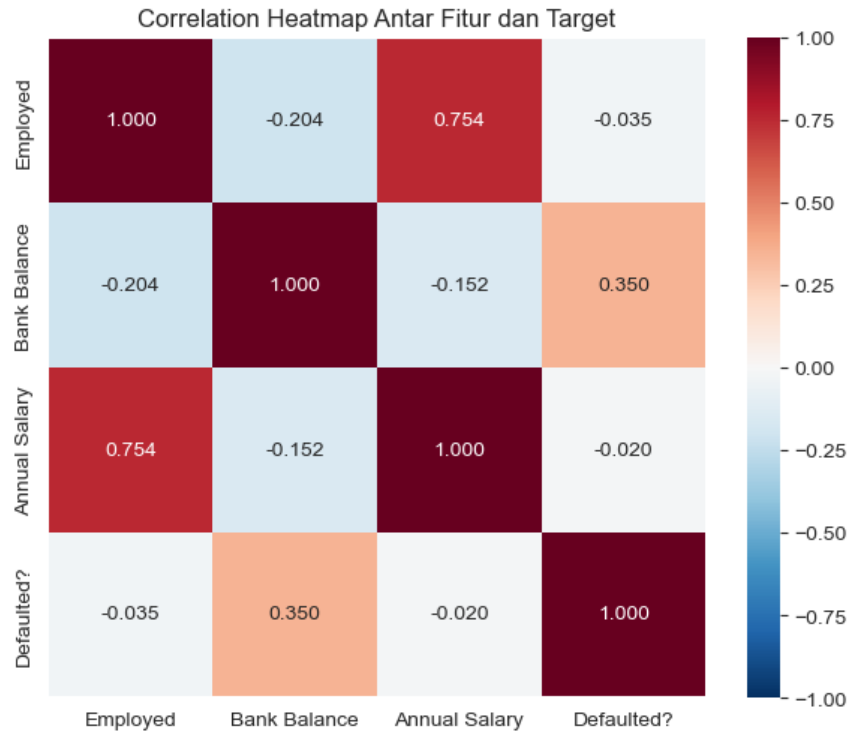
Gambar 3.2 Boxplot Bank Balance dan Annual Salary terhadap Defaulted?



Gambar 3.3 Distribusi (KDE) Bank Balance dan Annual Salary per Kelas

- **Bank Balance:** Terdapat perbedaan distribusi yang cukup kontras antara nasabah *default* dan tidak. Nasabah yang mengalami gagal bayar cenderung memiliki saldo tabungan yang secara signifikan lebih tinggi dibandingkan nasabah yang lancar. Hal ini mungkin terlihat berlawanan dengan intuisi awal, namun secara analitis memberikan wawasan bahwa saldo yang tinggi diiringi perilaku *default* bisa jadi menandakan pola penggunaan kredit yang tidak lazim.
- **Annual Salary:** Sebaliknya, distribusi *Annual Salary* tampak lebih tumpang tindih (*overlap*) antara kedua kelas. Ini mengindikasikan bahwa pendapatan tahunan mungkin bukan prediktor tunggal yang sekuat saldo bank dalam membedakan risiko *default*.

Selain distribusi fitur numerik, kami juga melakukan analisis *cross-tab* antara status pekerjaan (*Employed*) dan *Defaulted?* untuk melihat pola kategorikal. Hasilnya menunjukkan proporsi *default* pada nasabah yang tidak bekerja (4,31%) sedikit lebih tinggi dibandingkan nasabah yang bekerja (2,92%). Meskipun pengaruh status pekerjaan terlihat ada, perannya tidak sedominan pengaruh saldo bank. Perpaduan visualisasi ini menjadi landasan kuat mengapa rekayasa fitur (*feature engineering*) menjadi tahapan yang krusial sebelum data diumpankan ke dalam model. Untuk melengkapi gambaran ini, kami melakukan analisis korelasi antar fitur melalui *correlation heatmap* (Gambar 3.4) guna memetakan hubungan linear di antara variabel-variabel tersebut.



Gambar 3.4 Correlation Heatmap Antar Fitur dan Target

Correlation heatmap menunjukkan Bank Balance adalah satu-satunya fitur dengan korelasi cukup berarti terhadap Defaulted? ($r = 0,350$, arah positif), artinya nasabah dengan saldo bank yang lebih tinggi justru berasosiasi dengan risiko default yang lebih tinggi pula pada dataset ini, sebuah pola yang pada pandangan pertama terlihat berlawanan dengan intuisi umum, namun konsisten dan akan diperkuat lagi oleh hasil feature importance dan SHAP pada BAB V.5.8. Sementara itu, Employed dan Annual Salary saling berkorelasi tinggi satu sama lain ($r = 0,754$), namun keduanya lemah dalam membedakan status default secara individual ($r = -0,035$ dan $r = -0,020$).

3.6 Ringkasan 5 Insight Utama

Berdasarkan keseluruhan eksplorasi data yang telah dilakukan, terdapat lima temuan kunci yang menjadi landasan bagi strategi pemodelan selanjutnya:

- **Ketimpangan Kelas:** Distribusi data sangat timpang dengan kelas *default* hanya mencakup 3,33%. Kondisi ini menjadikan akurasi sebagai metrik yang menyesatkan (*misleading*), sehingga penggunaan F1-Score dan PR-AUC menjadi kewajiban mutlak.
- **Paradoks Bank Balance:** Berlawanan dengan intuisi umum, saldo bank berkorelasi positif dengan risiko *default* ($r = 0,350$). Nasabah dengan saldo lebih tinggi justru menunjukkan profil risiko yang lebih besar dalam dataset ini.
- **Peran Status Pekerjaan:** Meskipun nasabah yang tidak bekerja memiliki proporsi *default* lebih tinggi (4,31%) dibandingkan yang bekerja (2,92%), fitur *Employed* secara individual memiliki pengaruh yang sangat lemah terhadap target.

- **Informasi Fitur:** *Annual Salary* tidak memberikan pola pembeda yang tajam ($r = -0,020$). Hal ini mengindikasikan bahwa saldo riil jauh lebih informatif dalam memprediksi perilaku gagal bayar nasabah dibandingkan pendapatan tahunan.
- **Integritas Fitur:** Analisis korelasi menunjukkan tidak adanya multikolinearitas yang destruktif (di luar hubungan *Employed–Annual Salary*). Dengan demikian, seluruh fitur input tetap dipertahankan untuk memberikan konteks yang utuh bagi model.

3.7 Identifikasi Tantangan Dataset

Untuk menjawab tantangan yang ditemukan selama proses eksplorasi, strategi penanganan data telah dirancang dan dirangkum dalam tabel di bawah ini. Strategi ini akan diimplementasikan secara sistematis pada tahapan *pipeline* di BAB 4.

Tantangan	Deskripsi	Solusi yang Diterapkan
Imbalanced Data	Kelas default hanya 3,33% dari total data, risiko model bias ke kelas mayoritas	SMOTE pada training set, dibandingkan dengan <code>class_weight='balanced'</code> ; evaluasi dengan Precision/Recall/F1/PR-AUC (BAB IV.5 & BAB V)
Missing Values	Tidak ditemukan missing values pada dataset ini (hasil <code>isnull().sum()</code>)	Tetap dicek eksplisit sebagai langkah validasi standar pipeline
Outlier	31 baris outlier pada Bank Balance berdasarkan metode IQR	Outlier tidak dibuang otomatis karena berpotensi menjadi sinyal risiko yang relevan, bukan noise
Duplikasi Data	Tidak ditemukan baris duplikat (hasil <code>duplicated().sum()</code>)	Tetap dicek dengan <code>drop_duplicates()</code> sebagai langkah pencegahan
Keterbatasan Fitur	Hanya 3 fitur input mentah yang tersedia, membatasi kekayaan informasi	Feature engineering (Balance_to_Salary_Ratio, Balance_per_Employment, Salary_Bin) untuk memperkaya sinyal prediktif
Skala Fitur Berbeda	Bank Balance dan Annual Salary memiliki rentang nilai yang jauh berbeda	StandardScaler pada fitur numerik, fit hanya pada training set

BAB IV PERANCANGAN PIPELINE PEMBELAJARAN MESIN

4.1 Data Preprocessing

Tahapan ini dibagi menjadi dua proses utama untuk memastikan kualitas input data:

- **Tahap 1 — Data Cleaning:** Langkah awal dilakukan dengan menghapus kolom *Index* yang tidak memiliki nilai prediktif. Validasi integritas data dilakukan melalui pengecekan *missing values* dan baris duplikat yang, berdasarkan hasil observasi, tidak ditemukan dalam dataset. Terkait *outlier* pada fitur *Bank Balance*, keputusan diambil untuk tetap mempertahankan seluruh baris data. Hal ini didasari oleh justifikasi bisnis yang telah dibahas pada sub-bab 3.4, di mana nilai ekstrem tersebut dianggap sebagai sinyal risiko yang valid, bukan *noise* yang harus dieliminasi.
- **Tahap 2 — Feature Scaling:** Untuk mengatasi perbedaan skala antar variabel, *StandardScaler* diterapkan pada seluruh fitur numerik (*Bank Balance*, *Annual Salary*, serta fitur baru hasil rekayasa). Prinsip pencegahan *data leakage* diimplementasikan dengan sangat ketat: *scaling* dilakukan setelah proses pembagian data, di mana parameter penskalaan hanya dihitung berdasarkan *training set* dan kemudian diterapkan pada *validation* serta *test set*. Fitur kategorikal yang telah berbentuk biner (*Employed* dan hasil *one-hot encoding Salary Bin*) tidak dilibatkan dalam proses penskalaan ini untuk menjaga integritas nilai biner tersebut.

4.2 Feature Engineering

Guna memperkaya sinyal prediktif, tiga fitur baru dikonstruksi setelah proses pembagian data untuk menghindari kebocoran informasi:

- **Balance_to_Salary_Ratio:** Dihitung sebagai rasio antara *Bank Balance* dan *Annual Salary*. Fitur ini berfungsi sebagai indikator "bantalan finansial" yang mencerminkan proporsi kekayaan nasabah relatif terhadap pendapatannya.
- **Balance_per_Employment:** Fitur interaksi yang merepresentasikan perkalian antara saldo dan status pekerjaan. Tujuannya adalah untuk menangkap efek kombinasi antara stabilitas ekonomi (pekerjaan) dan akumulasi saldo nasabah.
- **Salary_Bin:** Variabel pendapatan tahunan dikonversi menjadi tiga kategori (rendah, menengah, tinggi) menggunakan *pd.cut*. Batasan bin dihitung secara eksklusif dari *training set* sebelum diterapkan ke subset lainnya, kemudian dilakukan *one-hot encoding* (*drop_first=True*) untuk memitigasi kolinearitas pada model.

4.3 Data Splitting Strategy

Strategi pembagian data dirancang untuk menjaga representasi kelas yang proporsional di setiap subset. Mengingat kondisi dataset yang sangat tidak seimbang, kami menerapkan *Stratified Split* dengan rasio 70:15:15 untuk *training*, *validation*, dan *test set*, dengan menetapkan parameter *random_state=42* untuk menjamin reproduksibilitas hasil eksperimen.

Lebih lanjut, untuk memastikan model memiliki stabilitas performa yang tinggi, proses pelatihan dilakukan dengan mengombinasikan *Stratified 5-Fold Cross-Validation* secara spesifik pada *training set*.

Pendekatan ini memungkinkan *tuning* hiperparameter dilakukan secara lebih objektif. Sementara itu, *validation set* digunakan sebagai pengecekan tambahan untuk memvalidasi konfigurasi model sebelum mencapai tahap evaluasi akhir. Sesuai dengan prinsip integritas riset, *test set* dijaga ketat agar tidak terpapar dalam proses pelatihan maupun *tuning* apa pun; subset ini hanya digunakan satu kali pada tahap akhir untuk mendapatkan estimasi performa model yang tidak bias pada data yang benar-benar baru. Tabel 4.1 berikut merangkum distribusi data dan fungsi spesifik dari masing-masing subset:

Subset	Proporsi	Jumlah Baris	Proporsi Kelas Default	Fungsi
Training Set	70%	7.000 baris (6.767 non-default, 233 default)	3,33%	Melatih model dan menerapkan teknik penanganan imbalance
Validation Set	15%	1.500 baris (1.450 non-default, 50 default)	3,33%	Sanity check tambahan sebelum evaluasi akhir
Test Set	15%	1.500 baris (1.450 non-default, 50 default)	3,33%	Evaluasi akhir model, tidak di-resample dan hanya dipakai satu kali

4.4 Penanganan Imbalanced Data

Inti dari penelitian ini terletak pada strategi penanganan ketimpangan kelas, yang menjadi tahapan paling krusial dalam membangun model yang andal. Untuk menguji efektivitas berbagai pendekatan, empat skenario dibandingkan secara sistematis dalam eksperimen ini: *baseline* (tanpa penanganan), *class weighting*, *SMOTE*, dan kombinasi *SMOTE* + *class weighting*. Strategi perbandingan ini dirancang untuk mengukur sejauh mana setiap teknik mampu memitigasi bias model terhadap kelas mayoritas.

Secara teknis, penerapan *SMOTE* dilakukan secara eksklusif pada *training set* setelah proses pembagian data. Hal ini krusial untuk mencegah kebocoran informasi (*data leakage*) dari sampel sintetis ke subset validasi maupun pengujian. Melalui teknik ini, distribusi kelas pada *training set* yang semula terdiri dari 6.767 nasabah *non-default* berbanding 233 nasabah *default*, berhasil diseimbangkan menjadi 6.767 berbanding 6.767 melalui pembangkitan 6.534 sampel sintetis baru. Detail mengenai teknik dan catatan penerapannya dirangkum dalam Tabel 4.2 berikut.

Teknik	Cara Kerja	Catatan Penerapan
Baseline (tanpa penanganan)	Model dilatih langsung pada distribusi kelas asli (29:1)	Dipakai sebagai pembanding untuk menunjukkan dampak buruk mengabaikan imbalance
Class Weighting	Memberi bobot lebih besar pada kesalahan klasifikasi kelas minoritas saat training, tanpa mengubah jumlah data	Diterapkan lewat parameter <code>class_weight='balanced'</code> pada model yang mendukungnya

Teknik	Cara Kerja	Catatan Penerapan
SMOTE	Membuat sampel sintetis baru untuk kelas minoritas dengan interpolasi linear antar titik minoritas dan k-NN terdekatnya di ruang fitur	Diterapkan hanya pada training set setelah split, agar tidak terjadi kebocoran data sintetis ke validation/test set
SMOTE + Class Weighting	Kombinasi kedua teknik di atas	Dibandingkan untuk melihat apakah kombinasi memberi peningkatan lebih lanjut dibanding SMOTE saja

4.5 Pipeline Workflow End-to-End

Sebagai penutup bab metodologi, Tabel 4.3 berikut menyajikan ringkasan alur kerja *pipeline* secara menyeluruh, mulai dari input data mentah hingga tahap *deployment* aplikasi Streamlit. Alur ini mencerminkan integrasi antara proses pemrosesan data, desain fitur, hingga pemilihan metrik evaluasi yang komprehensif untuk memastikan model tidak hanya akurat, tetapi juga dapat diinterpretasikan.

No	Tahap	Tools / Library	Output
1	Input Data	Pandas, Python	Raw DataFrame dari Default_Fin.csv (10000, 5)
2	Data Cleaning	Pandas	DataFrame bersih tanpa kolom Index, shape (10000, 4)
3	EDA	Matplotlib, Seaborn, Plotly	Visualisasi distribusi kelas, boxplot, KDE, correlation heatmap
4	Data Split	Scikit-learn (stratified train_test_split)	X_train (7000), X_val (1500), X_test (1500)
5	Feature Engineering	Pandas, NumPy	Balance_to_Salary_Ratio, Balance_per_Employment, Salary_Bin
6	Scaling	Scikit-learn (StandardScaler)	X_train_scaled, X_val_scaled, X_test_scaled
7	Imbalance Handling	Manual SMOTE / imbalanced-learn SMOTE, class_weight	X_train_resampled, y_train_resampled per skenario
8	Training & Tuning	Scikit-learn, XGBoost, GridSearchCV	Model terlatih dan ter-tuning (.pkl)
9	Evaluasi	Scikit-learn metrics, SHAP	Precision, Recall, F1, ROC-AUC, PR-AUC, Confusion Matrix, SHAP values
10	Output & Deployment	joblib, JSON, Streamlit	Model (.pkl), eval_artifacts.json, aplikasi Streamlit interaktif

BAB V STRATEGI PEMODELAN DAN EKSPERIMEN

5.1 Algoritma Machine Learning yang Digunakan

Eksperimen ini mengevaluasi tiga algoritma untuk mendapatkan model risiko kredit yang optimal:

- **Random Forest Classifier (Model Utama):** Dipilih sebagai model utama karena kemampuannya membangun banyak *decision tree* secara paralel melalui *Bootstrap Aggregating* (Bagging). Random Forest unggul dalam menangkap interaksi non-linear antar fitur tanpa asumsi distribusi tertentu, relatif tahan terhadap *outlier*, serta mampu menangani *class weight* secara native. Selain itu, model ini menyediakan *feature importance* yang krusial bagi interpretasi tim manajemen risiko kredit.
- **Logistic Regression (Model Pembanding):** Digunakan sebagai *baseline* linear sederhana untuk mengukur *uplift* performa yang diberikan oleh model berbasis *tree*. Model ini tetap relevan sebagai tolok ukur karena sifatnya yang stabil terhadap *overfitting* pada dataset dengan dimensi terbatas.
- **XGBoost (Model Pembanding):** Sebagai representasi keluarga algoritma *boosting*, XGBoost membangun *tree* secara sekuensial dengan fokus memperbaiki kesalahan prediksi dari iterasi sebelumnya. Kehadirannya memberikan komparasi mendalam antara paradigma *bagging* (Random Forest) dan *boosting*.

5.2 Rancangan Eksperimen

Eksperimen dilakukan secara sistematis untuk menjawab rumusan masalah mengenai efektivitas teknik penanganan *imbalance*. Empat skenario—*baseline*, *class weight*, *SMOTE*, serta kombinasi *SMOTE + class weight*—diuji silang pada ketiga model menggunakan *Stratified 5-Fold Cross-Validation* (*shuffle=True*, *random_state=42*) pada *training set*. Pendekatan ini memastikan reliabilitas hasil evaluasi terhadap lima metrik utama: F1, Average Precision (PR-AUC), Recall, Precision, dan ROC-AUC.

5.3 Metrik Evaluasi

Mengingat tingkat ketimpangan kelas yang ekstrem (rasio 29:1), akurasi tidak digunakan sebagai metrik utama karena potensi bias yang menyesatkan—di mana model yang hanya menebak kelas mayoritas tetap bisa mencapai akurasi 96,67%. Oleh karena itu, evaluasi berfokus pada metrik yang mampu mengukur performa pada kelas minoritas:

Metrik	Definisi	Relevansi
Precision	Proporsi prediksi default yang benar-benar default	Mengukur seberapa dapat dipercaya peringatan risiko yang diberikan model
Recall (Sensitivity)	Proporsi nasabah default aktual yang berhasil terdeteksi model	Metrik terpenting; kegagalan mendeteksi nasabah default (false negative) berdampak finansial paling besar

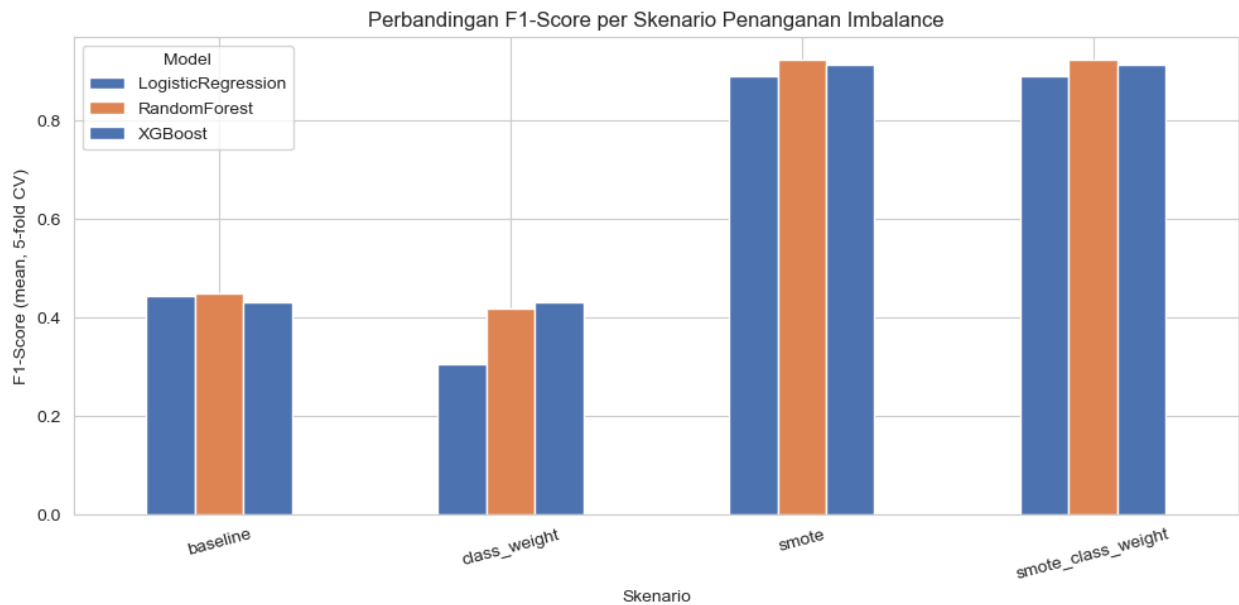
Metrik	Definisi	Relevansi
F1-Score	Harmonic mean antara Precision dan Recall	Menyeimbangkan trade-off antara kedua metrik di atas
ROC-AUC	Area di bawah kurva ROC (True Positive Rate vs False Positive Rate)	Mengukur kemampuan model membedakan kedua kelas di seluruh threshold
PR-AUC	Area di bawah kurva Precision-Recall	Lebih representatif dibanding ROC-AUC untuk kasus imbalance ekstrem seperti ini; dipakai sebagai scoring utama saat tuning
Confusion Matrix	Tabel True Positive, False Positive, True Negative, False Negative	Memberi gambaran menyeluruh jenis kesalahan model, terutama false negative pada nasabah berisiko

5.4 Hasil Eksperimen: Perbandingan Skenario Penanganan Imbalance (Cross-Validation)

Eksperimen terhadap dua belas kombinasi (3 model $\times$ 4 skenario) menggunakan *Stratified 5-Fold Cross-Validation* pada *training set* (7.000 baris nasabah) menunjukkan bahwa teknik penanganan ketimpangan kelas memberikan dampak yang sangat kontras terhadap performa model. Tanpa penanganan (*baseline*), *Recall* pada kelas *default* hanya berkisar di angka 0,32 hingga 0,36, meskipun nilai *ROC-AUC* tampak tinggi. Hal ini menegaskan bahwa *ROC-AUC* kurang sensitif terhadap kelas minoritas yang sangat kecil, sehingga penggunaan *PR-AUC* dan *F1-Score* menjadi lebih krusial sebagai indikator performa yang jujur.

Model	Skenario	F1 (mean)	PR-AUC (mean)	Recall (mean)	Precision (mean)	ROC-AUC (mean)
Random Forest	SMOTE	0,9225	0,9693	0,9369	0,9084	0,9744
Random Forest	SMOTE + class_weight	0,9224	0,9684	0,9379	0,9074	0,9740
XGBoost	SMOTE	0,9143	0,9648	0,9366	0,8931	0,9701
XGBoost	SMOTE + class_weight	0,9143	0,9648	0,9366	0,8931	0,9701
Logistic Regression	SMOTE	0,8893	0,9487	0,9060	0,8731	0,9534
Logistic Regression	SMOTE + class_weight	0,8893	0,9487	0,9060	0,8731	0,9534
Logistic Regression	Baseline	0,4434	0,5578	0,3176	0,7706	0,9473
Logistic Regression	class_weight	0,3048	0,5478	0,8796	0,1844	0,9467
XGBoost	Baseline	0,4317	0,4781	0,3304	0,6504	0,9315
XGBoost	class_weight	0,4317	0,4781	0,3304	0,6504	0,9315

Model	Skenario	F1 (mean)	PR-AUC (mean)	Recall (mean)	Precision (mean)	ROC-AUC (mean)
Random Forest	Baseline	0,4487	0,4024	0,3606	0,6040	0,8859
Random Forest	class_weight	0,4185	0,3995	0,3219	0,6135	0,8805



Gambar 5.1 Perbandingan F1-Score per Skenario Penanganan Imbalance (Cross-Validation)

Setelah penerapan *SMOTE*, terjadi peningkatan tajam pada *Recall* hingga melampaui angka 0,90 untuk seluruh model. Berdasarkan hasil yang tersaji pada Tabel 5.1, Random Forest dengan skenario *SMOTE* mencatatkan *PR-AUC* tertinggi (0,9693) di antara seluruh kombinasi yang diuji. Oleh karena itu, skenario *SMOTE* dipilih sebagai dasar untuk tahapan *tuning* hiperparameter pada Random Forest dan XGBoost, sementara untuk Logistic Regression, skenario *SMOTE* tetap dipertahankan karena memberikan hasil yang identik dengan kombinasi *SMOTE* + *class weighting*.

5.5 Hyperparameter Tuning

Tuning dilakukan menggunakan GridSearchCV dengan scoring average_precision (PR-AUC) dan cross-validation yang sama (Stratified 5-Fold), pada skenario *SMOTE* yang terbukti terbaik pada tahap sebelumnya.

Model	Search Space	Best Parameters	Best CV PR-AUC
Random Forest	n_estimators: [200, 400]; max_depth: [None, 8, 12]; min_samples_leaf: [1, 3, 5]	n_estimators=400, max_depth=None, min_samples_leaf=3	0,9702

Model	Search Space	Best Parameters	Best CV PR-AUC
XGBoost	n_estimators: [200, 400]; max_depth: [3, 5, 7]; learning_rate: [0,03, 0,05, 0,1]	n_estimators=200, max_depth=7, learning_rate=0,1	0,9669

Kedua konfigurasi hyperparameter hasil GridSearchCV ini persis sama dengan yang dipakai pada script produksi `train_model.py`, memastikan model yang di-deploy pada aplikasi Streamlit konsisten dengan hasil eksperimen pada notebook.

5.6 Validasi pada Validation Set

Sebelum evaluasi final di test set, model Random Forest terbaik dicek dulu performanya pada validation set yang juga belum pernah dilihat selama SMOTE, cross-validation, maupun tuning, sebagai sanity check tambahan.

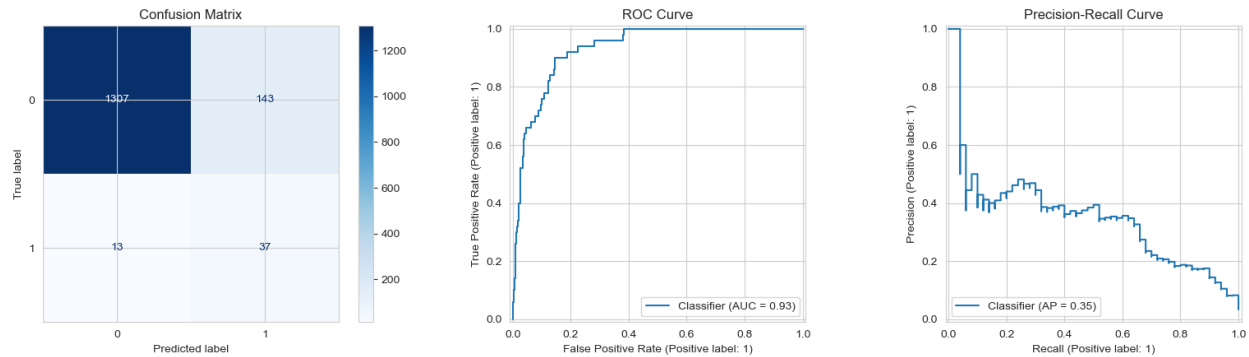
Metrik	Kelas 0 (Tidak Default)	Kelas 1 (Default)
Precision	0,9896	0,2264
Recall	0,9152	0,7200
F1-Score	0,9509	0,3445

Akurasi keseluruhan pada validation set adalah 0,9087, dengan ROC-AUC 0,9160 dan PR-AUC 0,3989. Hasil ini konsisten arahnya dengan evaluasi pada test set di subbab berikutnya, memperkuat keyakinan bahwa performa model stabil dan bukan kebetulan hasil satu kali split.

5.7 Evaluasi Model Final pada Test Set

Test set (1.500 baris, 50 nasabah *default*) dijaga kerahasiaannya dan tidak dilibatkan dalam proses *SMOTE*, *cross-validation*, maupun *tuning* untuk memastikan integritas evaluasi. Berdasarkan hasil yang disajikan pada Gambar 5.2, Random Forest mampu mencapai *Recall* sebesar 0,7400. Angka ini secara signifikan melampaui target minimal 0,70 yang ditetapkan pada tahap hipotesis (BAB II.5), membuktikan efektivitas model dalam menangkap sekitar 74% nasabah berisiko gagal bayar. Namun, *Precision* yang berada di angka 0,2056 mengindikasikan bahwa luaran model saat ini lebih tepat diposisikan sebagai alat bantu penyaringan awal (*early warning system*) yang tetap memerlukan verifikasi manual oleh analis kredit, dan bukan sebagai penentu keputusan akhir secara otomatis.

Metrik	Kelas 0 (Tidak Default)	Kelas 1 (Default)
Precision	0,9902	0,2056
Recall	0,9014	0,7400
F1-Score	0,9437	0,3217

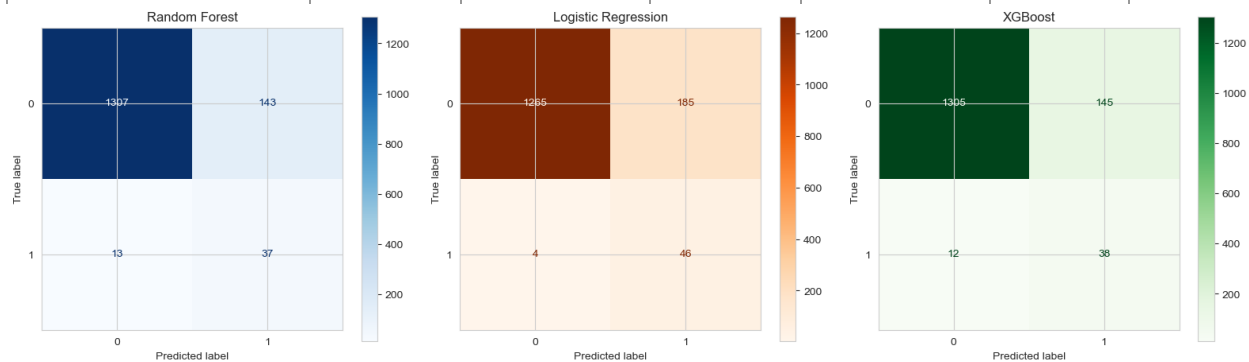


Gambar 5.2 Confusion Matrix, Kurva ROC, dan Kurva Precision-Recall Random Forest pada Test Set

5.7.1 Perbandingan Model Final pada Test Set (Random Forest vs Logistic Regression vs XGBoost)

Untuk memberikan perbandingan yang adil, Logistic Regression dan XGBoost dilatih ulang pada skenario terbaik masing-masing—yaitu *SMOTE*—lalu dievaluasi pada *test set* yang sama dengan Random Forest. Hasil komparasi pada Tabel 5.3 menunjukkan pola yang konsisten: seluruh model menghadapi *trade-off* antara *Recall* yang tinggi dan *Precision* yang rendah, sebuah konsekuensi logis dari rasio ketimpangan kelas yang mencapai 29:1.

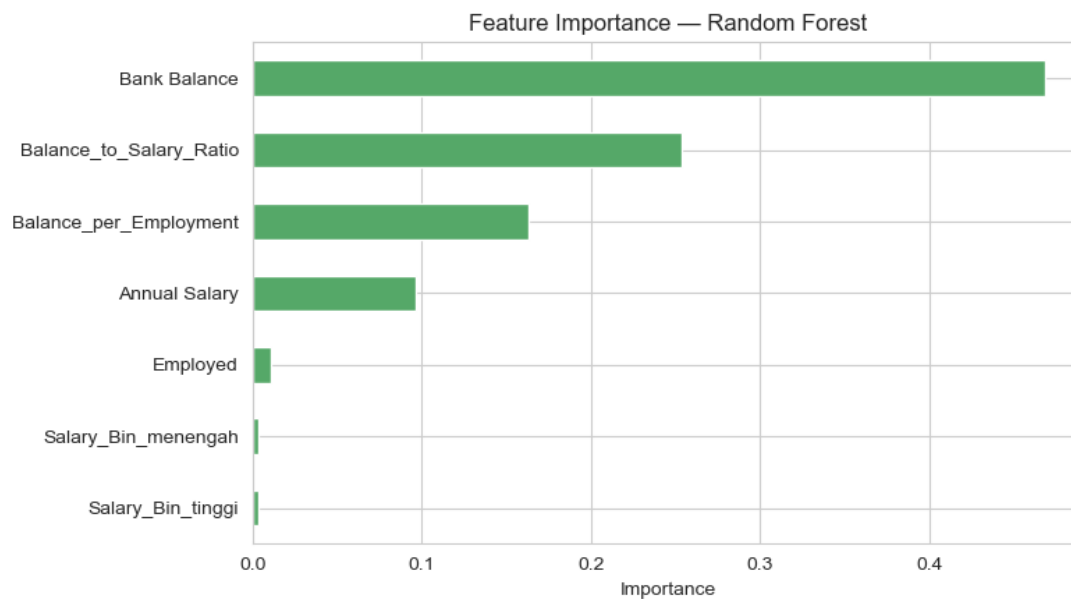
Model	Accuracy	Precision (kelas 1)	Recall (kelas 1)	F1-Score (kelas 1)	ROC-AUC	PR-AUC
Random Forest (SMOTE + tuning)	0,8960	0,2056	0,7400	0,3217	0,9315	0,3517
Logistic Regression (SMOTE)	0,8740	0,1991	0,9200	0,3274	0,9586	0,4830
XGBoost (SMOTE + tuning)	0,8953	0,2077	0,7600	0,3262	0,9311	0,3827



Gambar 5.3 Confusion Matrix Random Forest vs Logistic Regression vs XGBoost pada Test Set

Menariknya, Logistic Regression justru menunjukkan keunggulan pada metrik *ROC-AUC* dan *PR-AUC* dibandingkan kedua model berbasis *tree*. Di sisi lain, XGBoost memiliki performa *Recall* dan *Precision* yang sedikit di atas Random Forest, namun tertinggal dalam metrik kurva. Secara keseluruhan, perbedaan performa antar ketiga model tergolong sangat tipis (selisih di bawah 0,05). Hal ini mengonfirmasi temuan krusial bahwa pemilihan teknik penanganan *imbalance* (*SMOTE*) jauh lebih menentukan performa akhir dibandingkan pemilihan keluarga algoritma, sejalan dengan studi perbandingan algoritma untuk *credit scoring* pada data tidak seimbang oleh Brown dan Mues (2012).

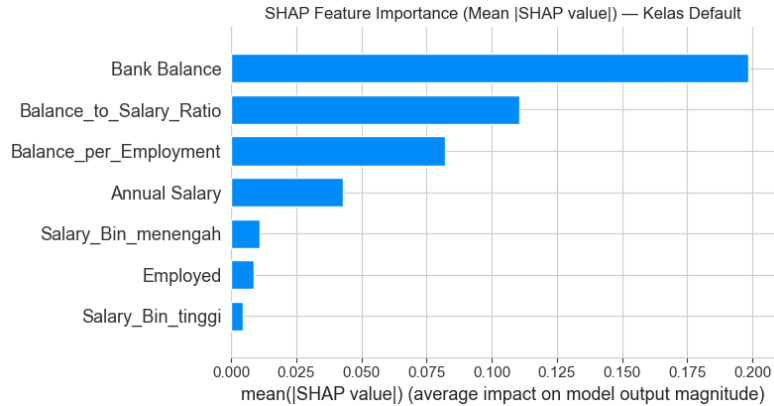
5.8 Feature Importance dan Interpretasi SHAP



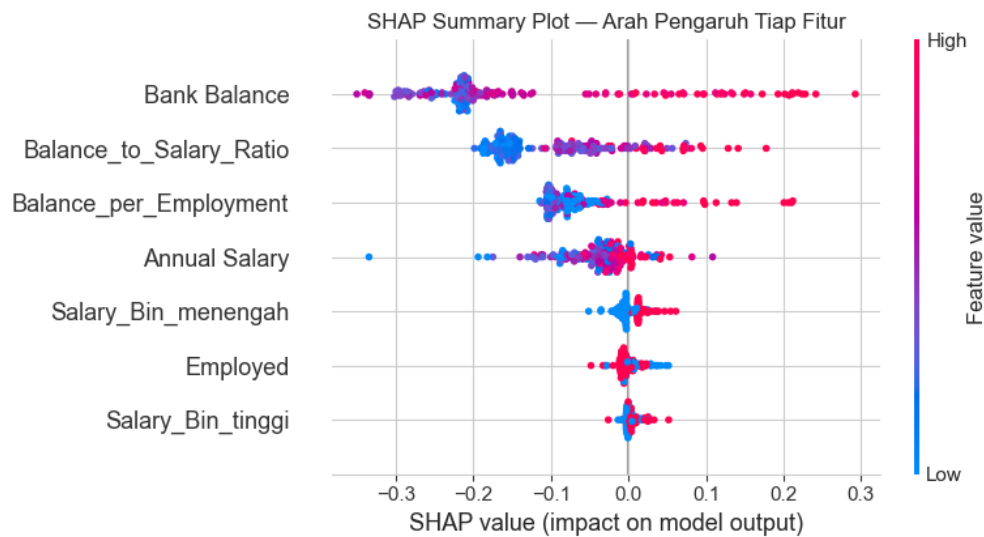
Gambar 5.4 Feature Importance Model Random Forest

Bank Balance menjadi fitur paling dominan dengan kontribusi 0,4682, disusul Balance_to_Salary_Ratio sebesar 0,2540 dan interaksi Balance_per_Employment sebesar 0,1632, sementara Annual Salary dan Employed masing-masing hanya berkontribusi 0,0968 dan 0,0112, dan kedua kolom hasil binning Salary_Bin bahkan di bawah 0,004. Temuan ini konsisten dengan hipotesis awal pada BAB II.5, bahwa rasio antara saldo dan pendapatan merupakan indikator risiko finansial yang lebih informatif dibanding nilai saldo atau gaji secara terpisah.

Feature importance bawaan Random Forest mengukur kontribusi rata-rata dalam mengurangi impuritas, tetapi tidak menunjukkan arah pengaruh tiap fitur. SHAP (SHapley Additive exPlanations), kerangka kerja pemersatu untuk interpretasi prediksi model berbasis teori Shapley value dari game theory (Lundberg & Lee, 2017), melengkapi ini dengan mengukur kontribusi tiap fitur secara individual per prediksi menggunakan TreeExplainer pada sampel 200 baris test set.



Gambar 5.5 SHAP Feature Importance (Mean Absolute SHAP Value) Kelas Default



Gambar 5.6 SHAP Summary Plot: Arah Pengaruh Tiap Fitur

SHAP bar plot mengonfirmasi urutan kepentingan fitur yang sama dengan feature importance bawaan. SHAP summary plot memberi informasi tambahan berupa arah pengaruh: nilai Bank Balance dan Balance_to_Salary_Ratio yang tinggi cenderung mendorong prediksi ke arah risiko default, sejalan dengan korelasi positif yang ditemukan pada BAB III.5 bahwa nasabah default pada dataset ini justru cenderung memiliki saldo nominal yang lebih tinggi, kemungkinan karena rasio saldo terhadap gaji atau pola pengeluaran yang tidak sehat, bukan karena kekurangan dana secara nominal.

5.9 Analisis Overfitting

Metrik	Train	Test	Selisih	Indikasi Overfitting (>0,15)
F1-Score	0,9646	0,3217	0,6429	Ya
Recall	0,9746	0,7400	0,2346	Ya

Selisih F1-Score dan Recall antara training dan test set berada jauh di atas ambang 0,15, mengindikasikan overfitting pada level evaluasi training set yang sudah di-SMOTE. Hal ini metodologis dan bukan berarti

model gagal generalisasi sepenuhnya, karena training set yang dipakai untuk menghitung metrik train sudah mengandung sampel sintetis SMOTE (yang secara inheren lebih mudah diprediksi oleh model yang sama yang menghasilkan pola interpolasinya), sementara test set murni data asli. Isu metodologis serupa secara spesifik dibahas oleh Santos dkk. (2018), yang mengingatkan bahwa skor cross-validation pada data yang di-resample sebelum splitting fold cenderung optimis. Konsistensi hasil antara validation set dan test set pada BAB V.5.6 dan V.5.7 (recall 0,72 vs 0,74, PR-AUC 0,3989 vs 0,3517) tetap menjadi bukti pendukung bahwa model generalisasi dengan wajar pada data yang benar-benar belum pernah dilihat.

5.10 Pemilihan Model Terbaik dan Justifikasi

Random Forest dengan skenario SMOTE dan hyperparameter hasil GridSearchCV dipilih sebagai model utama pada aplikasi deployment, dengan tiga pertimbangan. Pertama, Random Forest kompatibel dengan SHAP TreeExplainer dan feature importance berbasis pengurangan impuritas, memberi interpretasi yang lebih kaya dan lebih cepat dihitung dibanding koefisien linear Logistic Regression untuk kasus dengan interaksi fitur seperti Balance_per_Employment. Kedua, hasil validation set dan test set Random Forest konsisten satu sama lain, memperkuat keyakinan model generalisasi dengan wajar meskipun ada overfitting pada level perbandingan train-test mentah. Ketiga, dari sisi praktis, Recall 0,74 pada Random Forest dianggap sudah memadai untuk kebutuhan early warning tanpa membanjiri analis kredit dengan terlalu banyak nasabah yang perlu diverifikasi ulang seperti pada Logistic Regression yang mengejar Recall 0,92.

Meski begitu, perlu dicatat jujur bahwa Logistic Regression dengan PR-AUC dan ROC-AUC yang justru lebih tinggi (0,4830 dan 0,9586) adalah kandidat yang tetap layak dipertimbangkan apabila kebijakan bisnis memprioritaskan Recall setinggi mungkin, dan XGBoost tuning (PR-AUC 0,3827) juga tampil kompetitif sebagai pembanding kedua. Ketiga model tetap disertakan pada aplikasi Streamlit (BAB VII) agar pengguna dapat membandingkan langsung ketiganya, sementara Random Forest dipakai sebagai model utama pada halaman Model Demo karena pertimbangan interpretabilitas SHAP di atas.

BAB VI RENCANA DESAIN DAN IMPLEMENTASI DEPLOYMENT MODEL

6.1 Bentuk Deployment

Meskipun pada tahap proposal direncanakan penggunaan arsitektur dua lapisan (*backend* FastAPI dan *frontend* Streamlit), pada implementasi akhir arsitektur disederhanakan menjadi satu aplikasi Streamlit terpadu (app.py). Keputusan ini diambil untuk mengurangi kompleksitas *deployment* serta memfasilitasi proses penilaian (*grading*) yang lebih efisien tanpa mengurangi fungsionalitas yang diharapkan—mencakup visualisasi EDA, demonstrasi prediksi model, hingga interpretasi hasil berbasis SHAP. Pendekatan ini terbukti memadai untuk kebutuhan demonstrasi interaktif bagi analis kredit, dibandingkan dengan sistem produksi skala besar yang memerlukan manajemen API yang jauh lebih kompleks.

6.2 Model dan Artefak yang Digunakan

Seluruh model dan artefak pendukung dihasilkan melalui skrip `train_model.py` dan disimpan dalam folder `/models` menggunakan format `joblib` dan `json`. Seluruh konfigurasi hiperparameter yang digunakan merujuk pada hasil optimasi yang telah dibahas pada sub-bab 5.5. Detail mengenai artefak yang dihasilkan disajikan pada Tabel 6.1 berikut.

Artefak	Fungsi
<code>models/random_forest_model.pkl</code>	Model Random Forest final (<code>n_estimators=400</code> , <code>max_depth=None</code> , <code>min_samples_leaf=3</code>), model utama pada halaman Model Demo
<code>models/xgboost_model.pkl</code>	Model XGBoost final (<code>n_estimators=200</code> , <code>max_depth=7</code> , <code>learning_rate=0,1</code>), sebagai pembanding pada halaman Evaluasi Model
<code>models/scaler.pkl</code>	StandardScaler yang sudah di-fit pada training data untuk 4 kolom numerik
<code>models/feature_names.json</code>	Daftar nama dan urutan fitur final yang diharapkan model (7 kolom setelah feature engineering dan one-hot encoding)
<code>models/salary_bins.json</code>	Batas bin Annual Salary (rendah/menengah/tinggi) hasil <code>pd.qcut</code> pada training set, dipakai ulang saat preprocessing input baru
<code>models/eval_artifacts.json</code>	Seluruh metrik evaluasi, confusion matrix, kurva ROC/PR, dan feature importance ketiga model, dibaca langsung oleh halaman Evaluasi Model dan Interpretasi Hasil

Logistic Regression tidak disimpan sebagai file `.pkl` terpisah karena tidak digunakan untuk prediksi langsung pada halaman *Model Demo*, namun seluruh metrik evaluasinya tetap tersimpan lengkap dalam `eval_artifacts.json` sebagai bahan perbandingan pada halaman *Evaluasi Model*.

6.3 Alur Input → Preprocessing → Model → Output

Proses inferensi dalam aplikasi dirancang agar konsisten dengan *pipeline* pelatihan. Tabel 6.2 merinci alur kerja aplikasi saat menerima input dari analis kredit hingga menampilkan hasil prediksi.

Tahap	Komponen	Detail
Input	Form Streamlit (Model Demo)	Credit analyst memasukkan data nasabah: Employed (bekerja/tidak), Bank Balance, dan Annual Salary.
Preprocessing	Fungsi preprocess_input() pada app.py	Hitung Balance_to_Salary_Ratio dan Balance_per_Employment, tentukan Salary_Bin memakai salary_bins.json, reindex kolom sesuai feature_names.json, lalu transform dengan scaler.pkl.
Model	Random Forest (.pkl via joblib)	model.predict_proba(X_input) menghasilkan probabilitas risiko default (probabilitas kelas 1).
Interpretasi Lokal	SHAP TreeExplainer (shap.TreeExplainer)	Menghitung kontribusi tiap fitur terhadap prediksi spesifik nasabah tersebut, ditampilkan sebagai bar chart.
Output	Tampilan Streamlit	Badge risiko (LOW/MEDIUM/HIGH berdasarkan threshold 0,30 dan 0,60), probabilitas default dalam persen, dan grafik kontribusi fitur SHAP.

6.4 Struktur File Sistem dan Arsitektur

Dari sisi arsitektur, aplikasi beroperasi sebagai proses Python tunggal. Untuk efisiensi, penggunaan dekorator `@st.cache` diterapkan pada pemuatan model dan dataset, sehingga data tidak dimuat ulang setiap kali terjadi interaksi pengguna. Meskipun pendekatan ini memadai untuk kebutuhan demonstrasi, pemisahan arsitektur ke dalam lapisan API yang terpisah tetap menjadi opsi pengembangan lanjut yang valid untuk skenario produksi dengan intensitas akses pengguna yang tinggi di masa depan.

File / Folder	Fungsi
train_model.py	Script pelatihan: data cleaning, feature engineering, scaling, SMOTE, training RF/LR/XGBoost, evaluasi, ekspor seluruh artefak ke models/
app.py	Aplikasi Streamlit lima tab: Dashboard EDA, Model Demo, Evaluasi Model, Interpretasi Hasil, Dokumentasi
models/	Folder hasil ekspor train_model.py (lihat tabel BAB VI.2)
Default_Fin.csv	Dataset mentah, dipakai ulang oleh app.py pada halaman Dashboard EDA
requirements.txt	Dependensi: streamlit, pandas, numpy, scikit-learn, xgboost, joblib, shap, plotly

Dari sisi arsitektur, aplikasi ini bekerja sebagai satu proses Python tunggal (single-process Streamlit app): saat dijalankan, Streamlit memuat seluruh artefak model sekali di awal (memakai dekorator `@st.cache_resource` dan `@st.cache_data` agar tidak dimuat ulang setiap interaksi pengguna), kemudian merender salah satu dari lima halaman sesuai navigasi sidebar. Pendekatan ini cukup untuk kebutuhan demo interaktif satu pengguna pada konteks UAS, meskipun untuk skenario produksi dengan banyak pengguna bersamaan, pemisahan ke layer API terpisah seperti direncanakan pada proposal awal tetap menjadi opsi pengembangan lanjutan yang valid.

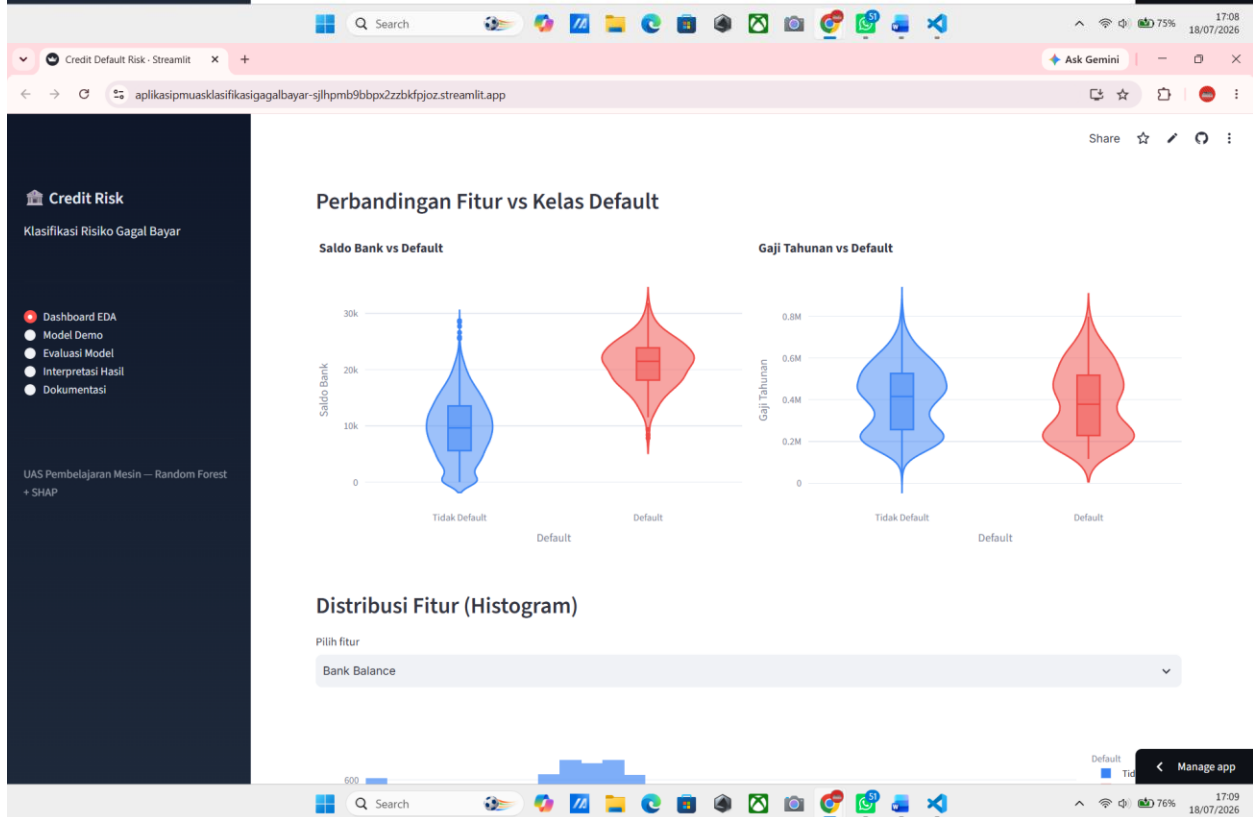
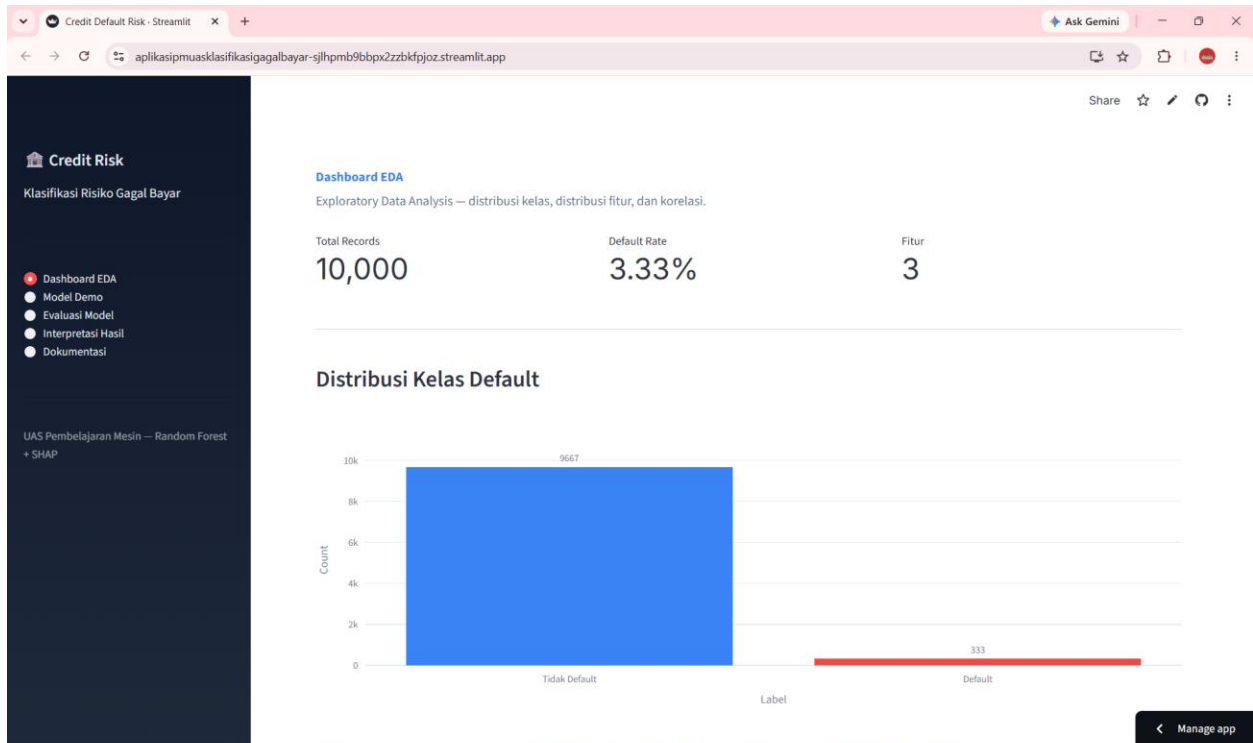
BAB VII DEPLOYMENT APLIKASI STREAMLIT

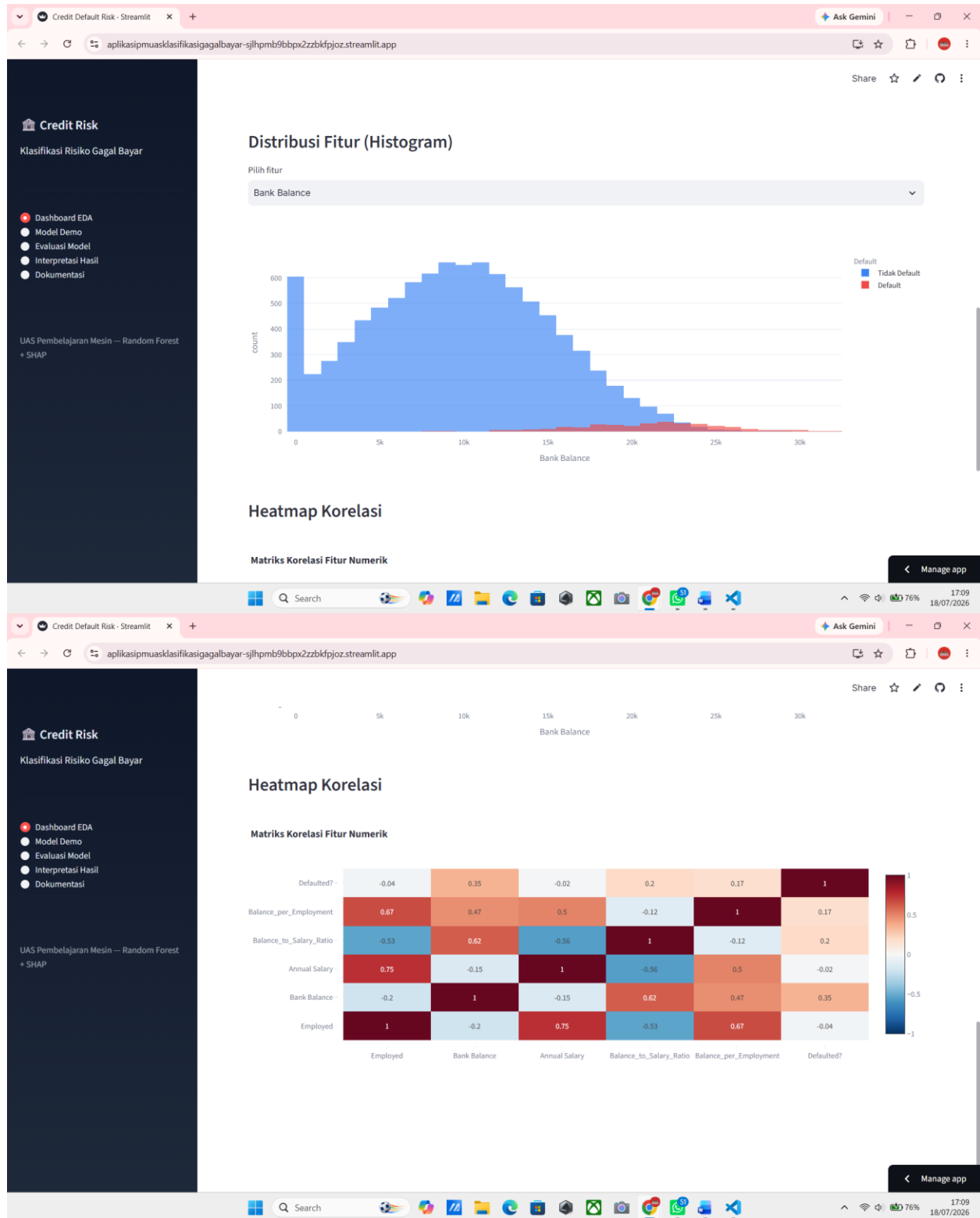
7.1 Ringkasan Aplikasi

Aplikasi web interaktif ini dikembangkan menggunakan *framework* Streamlit untuk memfasilitasi visualisasi data, demonstrasi model, serta interpretasi hasil prediktif secara *real-time*. Antarmuka aplikasi terdiri dari lima tab fungsional yang dapat diakses melalui navigasi *sidebar*: Dashboard EDA, Model Demo, Evaluasi Model, Interpretasi Hasil, dan Dokumentasi. Seluruh logika aplikasi tersimpan pada skrip `app.py`, sementara model dan artefak pendukung berasal dari hasil eksekusi `train_model.py` sebagaimana telah diuraikan pada BAB VI. Aplikasi telah berhasil di-*deploy* melalui Streamlit Community Cloud dan dapat diakses secara publik pada tautan berikut: <https://aplikasipmuasklasifikasigagalbayar-sjlhpmb9bbpx2zzbkfpjoz.streamlit.app/>.

7.2 Tab 1: Dashboard EDA

Halaman ini dirancang untuk memberikan gambaran eksploratif terhadap dataset. Sub-bab ini menampilkan metrik ringkasan (Total Records, Default Rate, dan jumlah fitur), distribusi kelas *Defaulted?*, *violin plot* untuk variabel *Bank Balance* dan *Annual Salary*, serta histogram interaktif dan *heatmap* korelasi antar fitur. Seluruh visualisasi menggunakan pustaka Plotly yang memungkinkan pengguna melakukan interaksi seperti *zoom* dan *hover* untuk analisis data yang lebih mendalam.

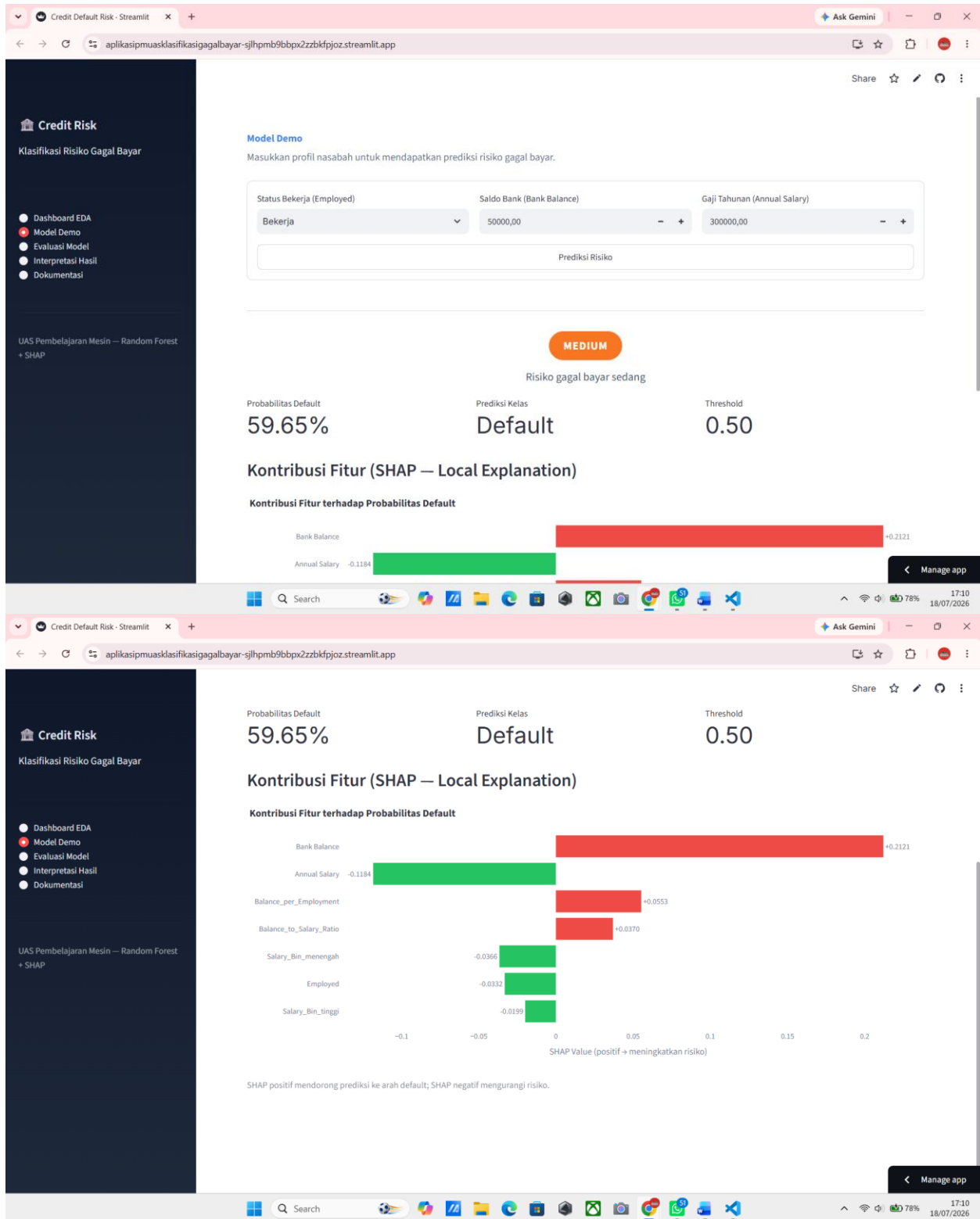




Gambar 7.1 Tampilan Tab Dashboard EDA pada Aplikasi yang Telah Di-deploy

7.3 Tab 2: Model Demo

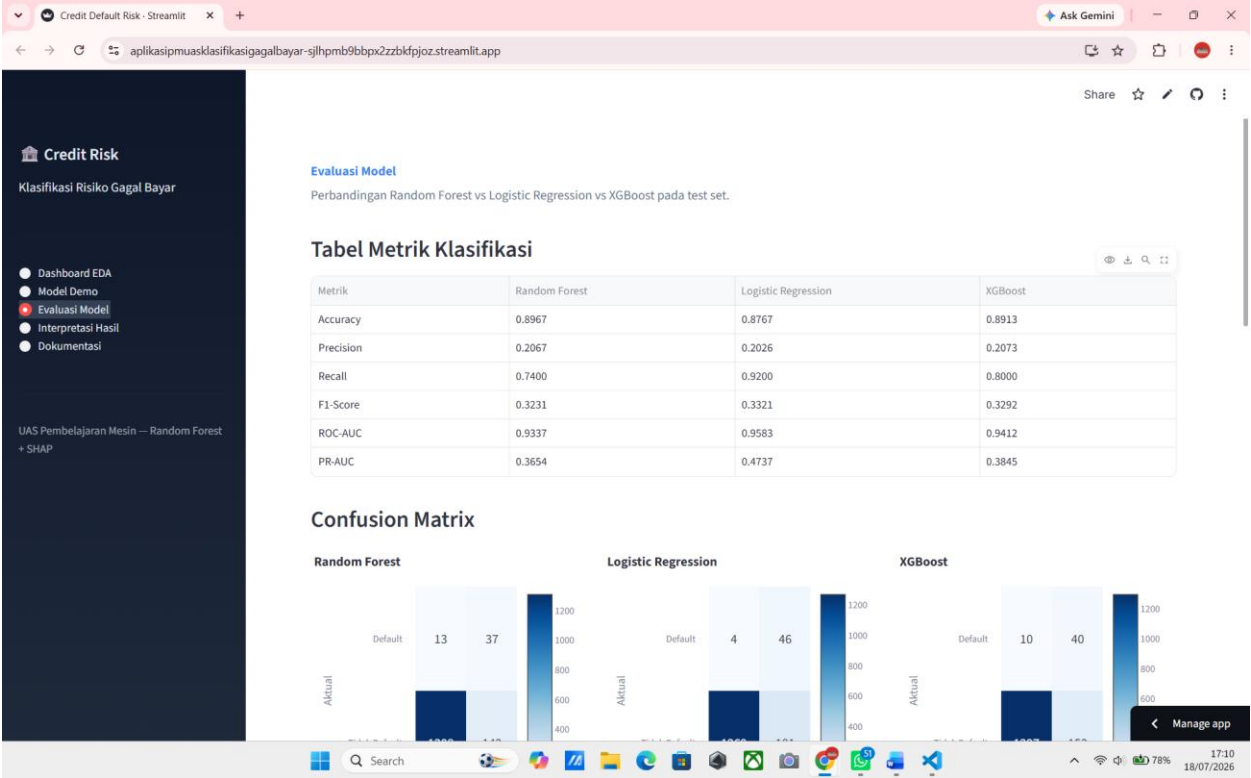
Halaman ini menyediakan antarmuka interaktif bagi pengguna untuk melakukan simulasi prediksi. Melalui form input status pekerjaan, saldo bank, dan gaji tahunan, model akan memproses data tersebut dan menampilkan status risiko (*LOW/MEDIUM/HIGH*), probabilitas *default*, serta grafik kontribusi fitur SHAP lokal. Fitur ini memungkinkan analisis kredit untuk memahami dasar pengambilan keputusan model pada kasus nasabah spesifik.

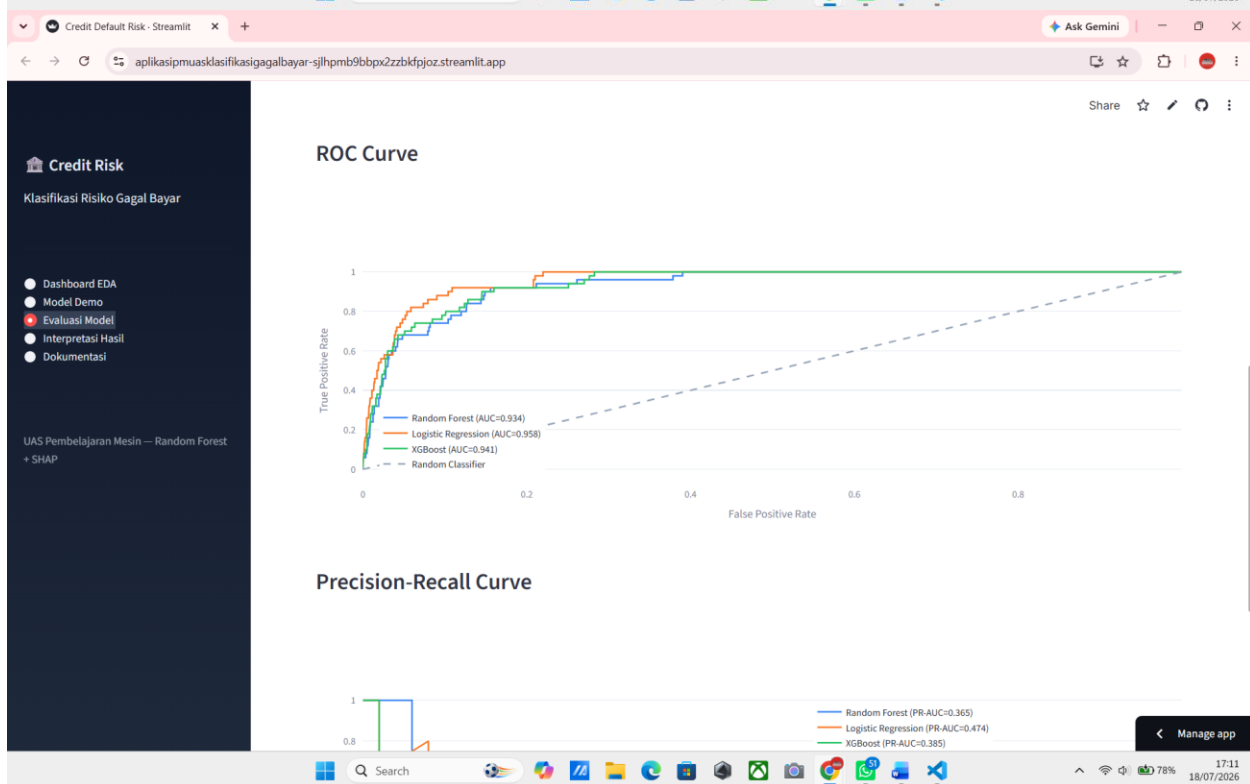
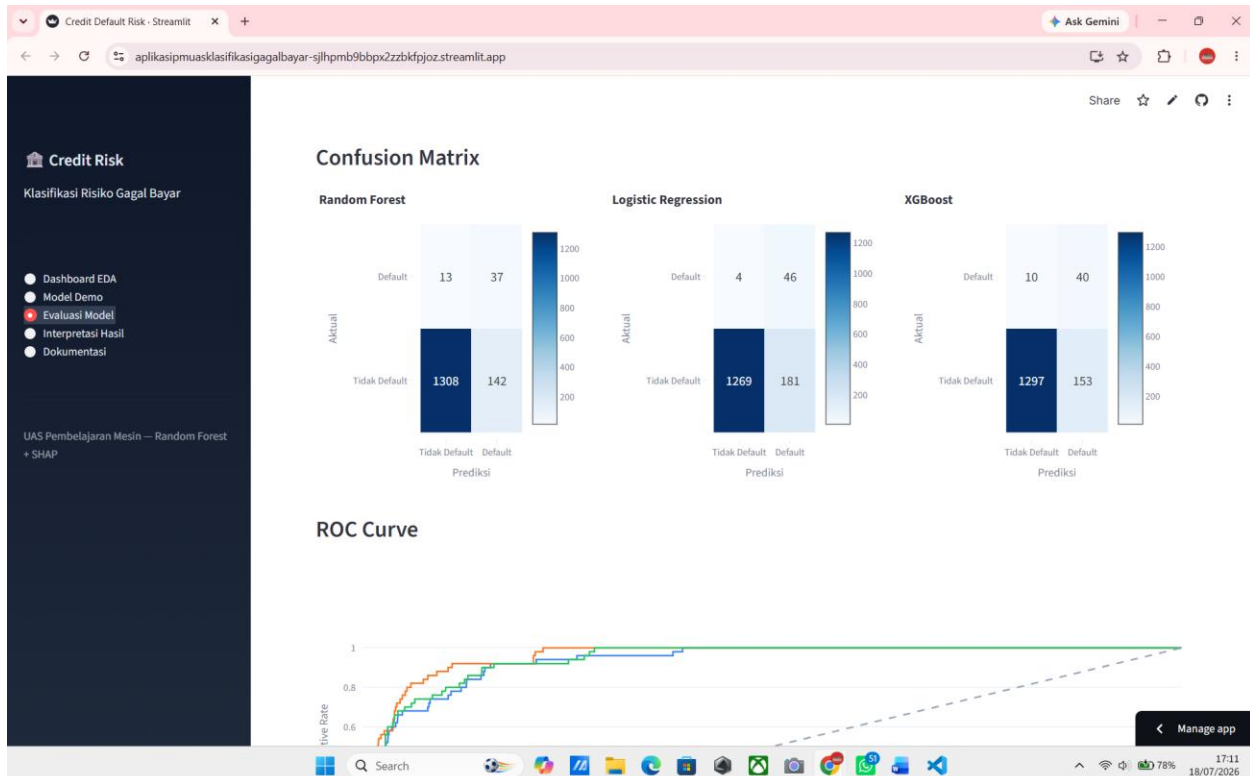


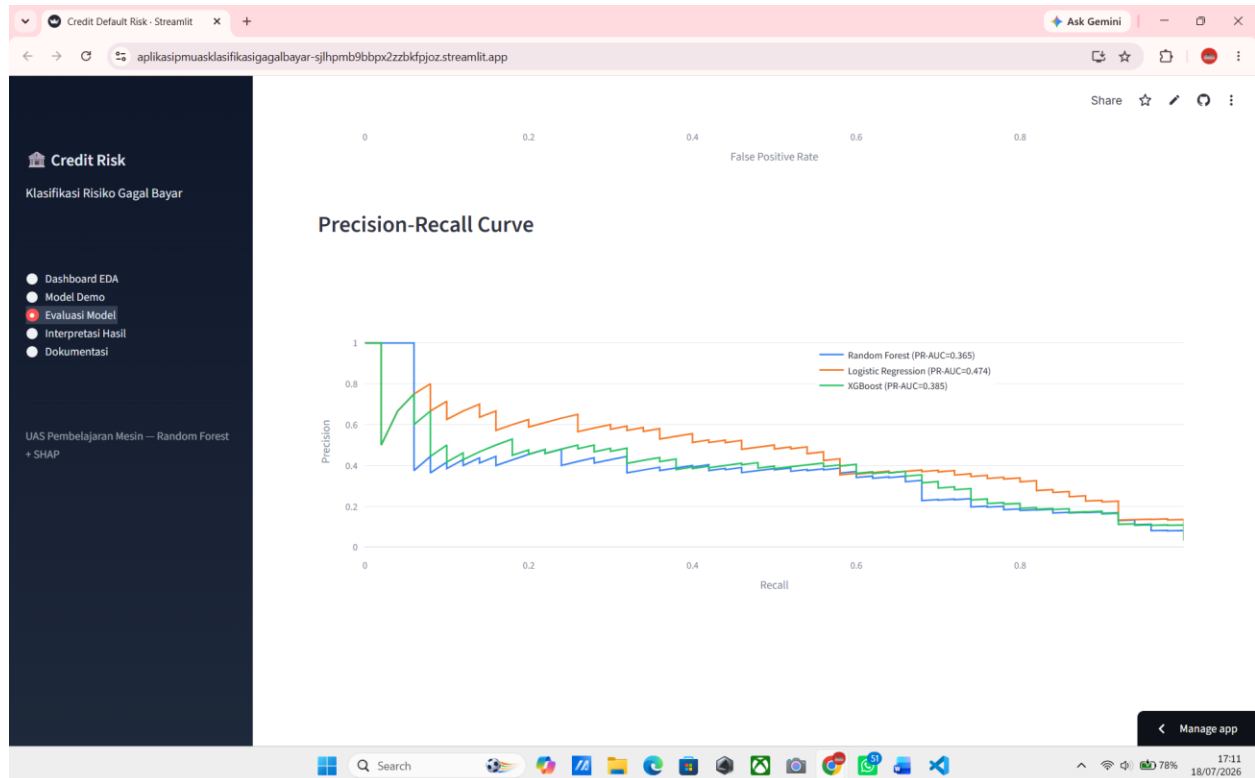
Gambar 7.2 Tampilan Tab Model Demo pada Aplikasi yang Telah Di-deploy

7.4 Tab 3: Evaluasi Model

Sub-bab ini menyajikan transparansi performa model melalui tabel perbandingan metrik utama (*Accuracy*, *Precision*, *Recall*, *F1-Score*, *ROC-AUC*, dan *PR-AUC*). Selain itu, halaman ini menampilkan *confusion matrix* serta kurva ROC dan *Precision-Recall* gabungan ketiga model. Data yang ditampilkan bersumber langsung dari eval_artifacts.json, memastikan konsistensi hasil evaluasi dengan analisis pada BAB V.5.7.



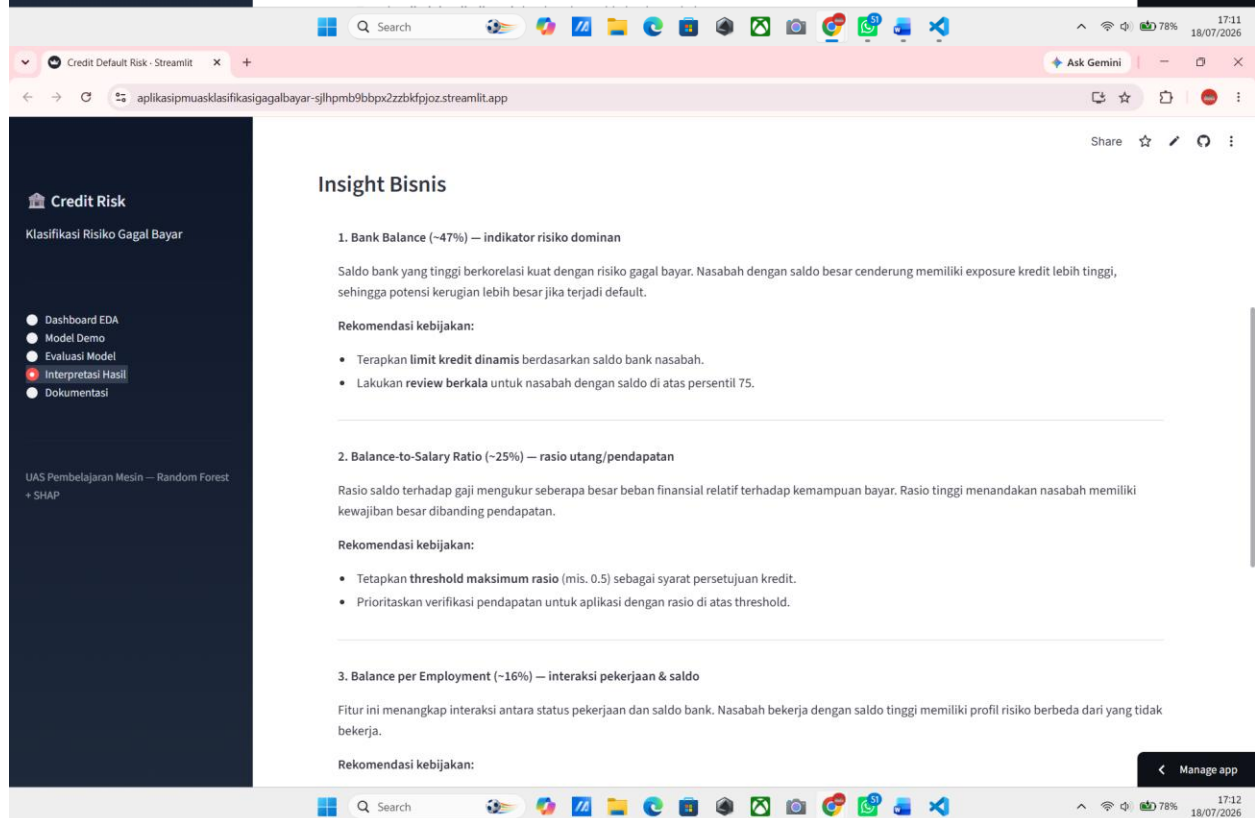
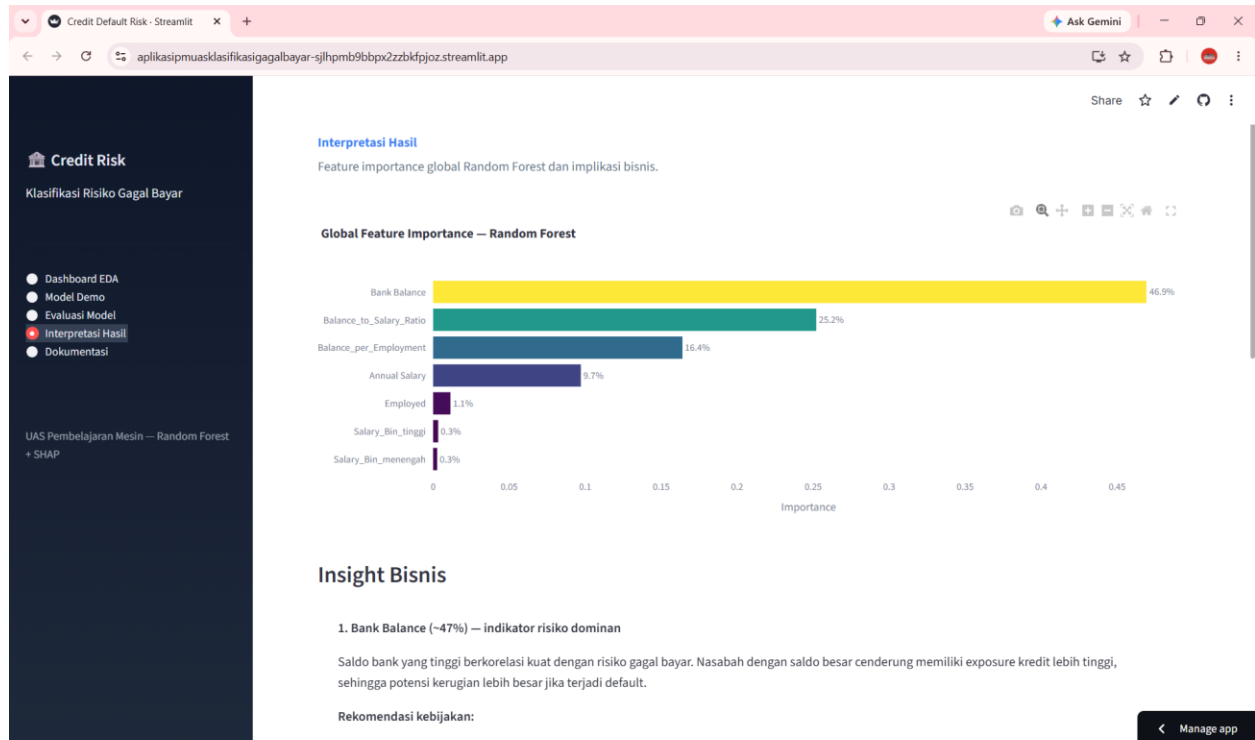


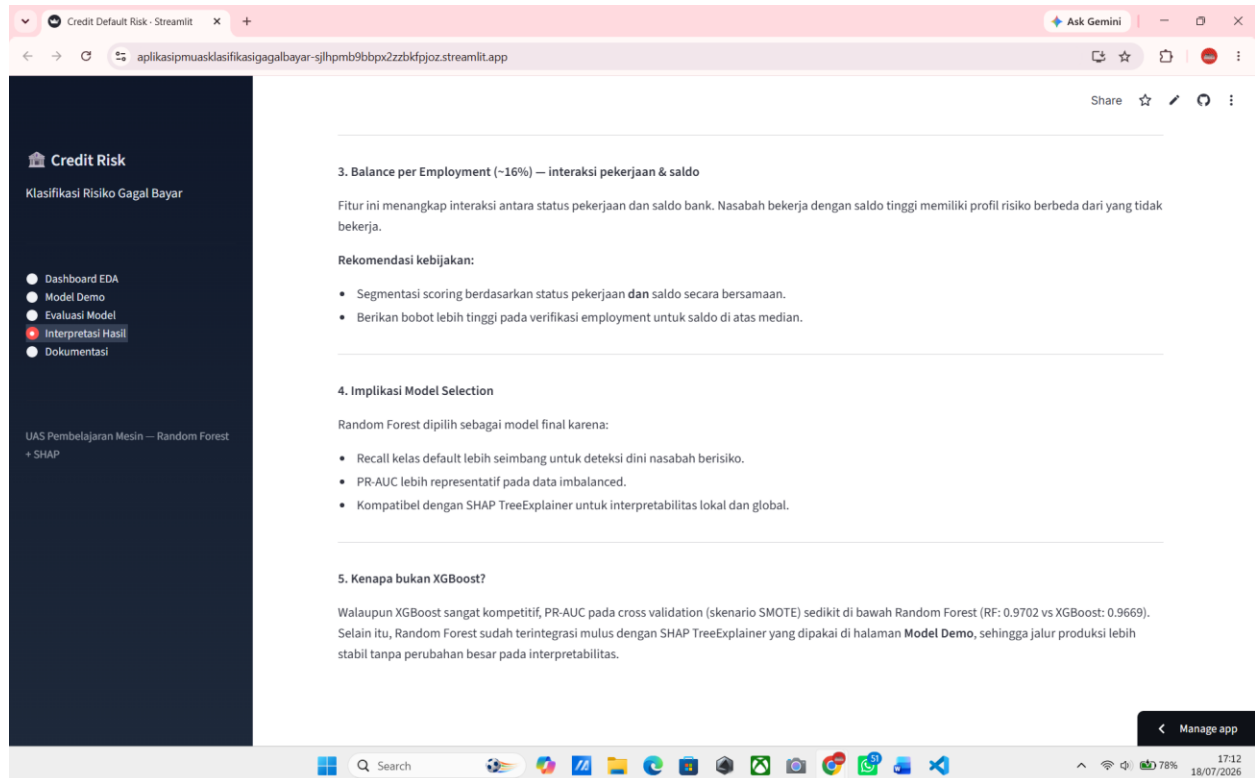


Gambar 7.3 Tampilan Tab Evaluasi Model pada Aplikasi yang Telah Di-deploy

7.5 Tab 4: Interpretasi Hasil

Halaman ini berfokus pada interpretasi model secara global menggunakan *feature importance* Random Forest. Insight bisnis yang disajikan mencakup peran dominan *Bank Balance*, rasio beban finansial melalui *Balance-to-Salary Ratio*, serta pengaruh interaksi status pekerjaan. Selain itu, terdapat rangkuman rekomendasi kebijakan kredit dan justifikasi pemilihan Random Forest sebagai model utama karena keunggulannya dalam aspek interpretabilitas dibandingkan model lain.





Gambar 7.4 Tampilan Tab Interpretasi Hasil pada Aplikasi yang Telah Di-deploy

7.6 Tab 5: Dokumentasi

Halaman ini berisi tiga sub-tab: Sumber Dataset (deskripsi atribut dan referensi dataset), Metodologi (ringkasan pipeline dari data cleaning hingga risk tiering), dan Panduan Pengguna (langkah-langkah penggunaan aplikasi untuk pengguna non-teknis, dari pengisian form hingga interpretasi badge risiko LOW/MEDIUM/HIGH)

Credit Default Risk - Streamlit

aplikasipmuasklasifikasigagalbayar-sjlhpmb9bbpx2zzbkfjjoz.streamlit.app

Ask Gemini

Share ☆ ↺ ⋮

Credit Risk

Klasifikasi Risiko Gagal Bayar

● Dashboard EDA

● Model Demo

● Evaluasi Model

● Interpretasi Hasil

● Dokumentasi

UAS Pembelajaran Mesin — Random Forest + SHAP

Dokumentasi

Sumber data, metodologi, dan panduan penggunaan aplikasi.

Sumber Dataset

Metodologi

Panduan Pengguna

Sumber Dataset

Dataset Default of Credit Card Clients berasal dari UCI Machine Learning Repository, yang dikompilasi dari data kredit konsumen di Taiwan.

Atribut	Deskripsi
Employed	Status pekerjaan (1 = bekerja, 0 = tidak)
Bank Balance	Saldo bank nasabah
Annual Salary	Gaji tahunan
Defaulted?	Label target (1 = gagal bayar, 0 = tidak)

Referensi:

- UCI ML Repository: [Default of Credit Card Clients](#)
- Kaggle: [Credit Card Default Prediction](#)

Manage app

17:12 18/07/2026

Credit Default Risk - Streamlit

aplikasipmuasklasifikasigagalbayar-sjlhpmb9bbpx2zzbkfjjoz.streamlit.app

Ask Gemini

Share ☆ ↺ ⋮

Credit Risk

Klasifikasi Risiko Gagal Bayar

● Dashboard EDA

● Model Demo

● Evaluasi Model

● Interpretasi Hasil

● Dokumentasi

UAS Pembelajaran Mesin — Random Forest + SHAP

Dokumentasi

Sumber data, metodologi, dan panduan penggunaan aplikasi.

Sumber Dataset

Metodologi

Panduan Pengguna

Metodologi Pipeline ML

1. Data Cleaning

- Drop kolom `Index` (Identifier).
- Hapus baris duplikat.

2. Train-Validation-Test Split

- Stratified split 70% / 15% / 15% (`random_state=42`).

3. Feature Engineering

- `Balance_to_Salary_Ratio` = Bank Balance / Annual Salary
- `Balance_per_Employment` = Bank Balance × Employed
- `Salary_Bin` — quantile binning (3 bins) dari training set, one-hot encoded

4. Feature Scaling

- `StandardScaler` pada fitur numerik; fitur biner tidak di-scale.

5. Class Imbalance — SMOTE

- Synthetic Minority Over-sampling Technique pada training set.
- Fallback manual SMOTE jika `imbalanced-learn` tidak tersedia.

Manage app

17:13 18/07/2026

Credit Default Risk - Streamlit

aplikasipmuasklasifikasigagalbayar-sjlhpmb9bbpx2zzbkfjz.streamlit.app

Ask Gemini

Share ☆ ↻ ⋮

Credit Risk

Klasifikasi Risiko Gagal Bayar

● Dashboard EDA

● Model Demo

● Evaluasi Model

● Interpretasi Hasil

● Dokumentasi

UAS Pembelajaran Mesin — Random Forest + SHAP

3. Feature Engineering

- Balance_to_Salary_Ratio = Bank Balance / Annual Salary
- Balance_per_Employment = Bank Balance × Employed
- Salary_Bin — quantile binning (3 bins) dari training set, one-hot encoded

4. Feature Scaling

- StandardScaler pada fitur numerik; fitur biner tidak di-scale.

5. Class Imbalance — SMOTE

- Synthetic Minority Over-sampling Technique pada training set.
- Fallback manual SMOTE jika imbalanced-learn tidak tersedia.

6. Model Training

- Random Forest: n_estimators=400, max_depth=None, min_samples_leaf=3
- Logistic Regression: baseline dengan max_iter=1000

7. Model Selection

- Random Forest dipilih berdasarkan PR-AUC, recall kelas default, dan interpretabilitas SHAP.

8. Risk Tiering

Tier	Probabilitas	Warna
LOW	< 0.30	Hijau
MEDIUM	0.30 – 0.60	Oranye
HIGH	> 0.60	Merah

Manage app

17:13 18/07/2026

Credit Default Risk - Streamlit

aplikasipmuasklasifikasigagalbayar-sjlhpmb9bbpx2zzbkfjz.streamlit.app

Ask Gemini

Share ☆ ↻ ⋮

Credit Risk

Klasifikasi Risiko Gagal Bayar

● Dashboard EDA

● Model Demo

● Evaluasi Model

● Interpretasi Hasil

● Dokumentasi

UAS Pembelajaran Mesin — Random Forest + SHAP

Dokumentasi

Sumber data, metodologi, dan panduan penggunaan aplikasi.

Sumber DatasetMetodologiPanduan Pengguna

Panduan Pengguna (Non-Teknis)

Langkah 1 — Buka halaman "Model Demo" Pilih menu Model Demo di sidebar kiri.

Langkah 2 — Isi profil nasabah

- Status Bekerja: pilih "Bekerja" atau "Tidak Bekerja".
- Saldo Bank: masukkan saldo rekening nasabah (angka).
- Gaji Tahunan: masukkan pendapatan tahunan nasabah.

Langkah 3 — Klik "Prediksi Risiko" Sistem akan menampilkan:

- Badge risiko (LOW / MEDIUM / HIGH) dengan warna.
- Probabilitas default dalam persen.
- Grafik SHAP — fitur mana yang paling mempengaruhi prediksi.

Langkah 4 — Interpretasi hasil

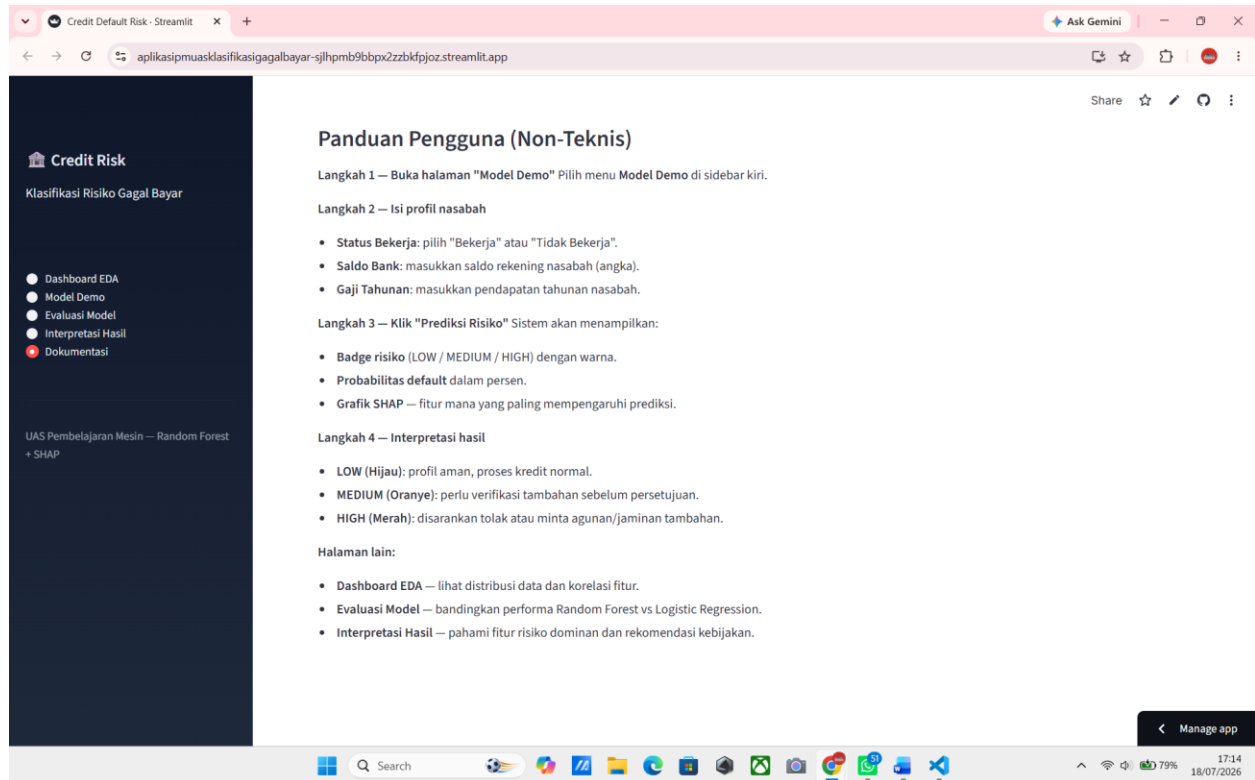
- LOW (Hijau): profil aman, proses kredit normal.
- MEDIUM (Oranye): perlu verifikasi tambahan sebelum persetujuan.
- HIGH (Merah): disarankan tolak atau minta agunan/jaminan tambahan.

Halaman lain:

- Dashboard EDA — lihat distribusi data dan korelasi fitur.
- Evaluasi Model — bandingkan performa Random Forest vs Logistic Regression.

Manage app

17:14 18/07/2026



Gambar 7.5 Tampilan Tab Dokumentasi pada Aplikasi yang Telah Di-deploy

7.7 Keluaran Wajib Soal 4

Tabel 7.1 berikut merangkum pemenuhan seluruh persyaratan fungsional dan keluaran wajib yang ditetapkan dalam instruksi tugas.

Keluaran Wajib	Status	Keterangan
Aplikasi Streamlit yang di-deploy	Terpenuhi	Sudah di-deploy ke Streamlit Community Cloud, tautan tersedia pada subbab 7.1
Kode aplikasi lengkap (app.py, model pickle, requirements.txt)	Terpenuhi	app.py dan train_model.py tersedia; artefak model dihasilkan otomatis pada folder models/, dijelaskan pada BAB VI
Screenshot antarmuka aplikasi	Terpenuhi	Sisipkan tangkapan layar asli pada subbab 7.2 hingga 7.6 di atas menggantikan placeholder

BAB VIII TIMELINE PROYEK

Rencana pelaksanaan proyek dibagi ke dalam delapan fase selama delapan minggu. Pada Ujian Tengah Semester sebelumnya, fokus utama berada pada Fase 1 hingga Fase 4, yaitu perumusan masalah, analisis data, dan perancangan pipeline. Memasuki Ujian Akhir Semester ini, cakupan proyek dilanjutkan hingga Fase 5 sampai Fase 8, mencakup eksperimen perbandingan skenario imbalance, tuning hyperparameter, interpretasi SHAP, deployment aplikasi Streamlit, hingga penyelesaian laporan akhir.

Fase	Kegiatan	Minggu
Fase 1	Persiapan & EDA — eksplorasi awal data, cek distribusi kelas Defaulted?, visualisasi ketimpangan kelas dan korelasi antar fitur	Minggu 1
Fase 2	Preprocessing — data cleaning (drop Index, cek missing values & duplikat), feature engineering (Balance_to_Salary_Ratio, Balance_per_Employment, Salary_Bin), scaling	Minggu 2
Fase 3	Eksperimen Imbalance Handling — implementasi dan perbandingan baseline, class_weight, SMOTE, dan kombinasinya pada 3 model memakai Stratified 5-Fold CV	Minggu 3
Fase 4	Modeling & Hyperparameter Tuning — training Random Forest, Logistic Regression, XGBoost baseline, dilanjutkan GridSearchCV untuk RF dan XGBoost	Minggu 4
Fase 5	Analisis & Evaluasi — evaluasi final pada validation dan test set dengan Precision/Recall/F1/ROC-AUC/PR-AUC, analisis confusion matrix, feature importance, dan SHAP	Minggu 5
Fase 6	Analisis Overfitting & Pemilihan Model — perbandingan train vs test, penetapan Random Forest sebagai model utama dengan justifikasi lengkap	Minggu 6
Fase 7	Development & Deployment — ekspor model ke .pkl dan JSON lewat train_model.py, bangun aplikasi Streamlit lima tab (app.py), deploy ke Streamlit Community Cloud	Minggu 7
Fase 8	Testing, Dokumentasi & Finalisasi — pengujian end-to-end aplikasi, pengambilan screenshot, penulisan laporan akhir, finalisasi deliverable	Minggu 8

BAB IX KESIMPULAN DAN REKOMENDASI

9.1 Kesimpulan

Berdasarkan serangkaian eksperimen yang telah dilakukan, dapat ditarik beberapa kesimpulan utama:

- Dataset Default_Fin.csv terbukti mengandung ketidakseimbangan kelas yang sangat signifikan pada variabel Defaulted?, dengan rasio 29 banding 1 (3,33% default berbanding 96,67% tidak default), tanpa missing values maupun baris duplikat.
- Penerapan SMOTE terbukti krusial di ketiga keluarga algoritma yang diuji. Recall kelas minoritas meningkat dari sekitar 0,32–0,36 pada skenario baseline (tanpa penanganan) menjadi di atas 0,90 pada skenario SMOTE untuk Random Forest, Logistic Regression, maupun XGBoost saat cross-validation.
- Pada evaluasi test set akhir, Random Forest (SMOTE + tuning) mencapai Recall 0,7400 dan PR-AUC 0,3517; Logistic Regression (SMOTE) mencapai Recall 0,9200 dan PR-AUC 0,4830 (tertinggi); XGBoost (SMOTE + tuning) mencapai Recall 0,7600 dan PR-AUC 0,3827 — ketiganya menunjukkan performa yang sebanding, mengonfirmasi bahwa penanganan class imbalance jauh lebih menentukan performa dibanding pemilihan keluarga algoritma.
- Feature importance Random Forest dan interpretasi SHAP secara konsisten mengidentifikasi Bank Balance, Balance_to_Salary_Ratio, dan Balance_per_Employment sebagai tiga fitur paling berpengaruh terhadap risiko default, dengan arah pengaruh positif (saldo dan rasio yang lebih tinggi berasosiasi dengan risiko default yang lebih tinggi pada dataset ini).
- Model utama berhasil diimplementasikan ke dalam aplikasi web interaktif berbasis Streamlit dengan lima tab fungsional, memungkinkan pengguna non-teknis melakukan eksplorasi data, mencoba prediksi, meninjau evaluasi model, dan memahami interpretasi hasil tanpa perlu membaca kode program.

9.2 Rekomendasi

- Melakukan pengumpulan fitur tambahan di luar tiga fitur mentah yang tersedia (misalnya riwayat kredit, jumlah tanggungan, atau jenis pinjaman), mengingat keterbatasan jumlah fitur menjadi salah satu batasan utama proyek ini.
- Mengeksplorasi ambang batas keputusan (decision threshold) selain 0,50 secara sistematis, misalnya lewat analisis kurva Precision-Recall pada BAB V.5.7, untuk mencari titik optimal sesuai kebijakan risiko bisnis yang diinginkan, alih-alih memakai threshold default 0,50.
- Mempertimbangkan teknik penanganan imbalance tambahan seperti ADASYN atau kombinasi SMOTE dengan undersampling (SMOTE-Tomek), sebagai pembanding terhadap SMOTE murni yang telah diterapkan pada proyek ini.
- Melengkapi aplikasi Streamlit dengan opsi bagi pengguna untuk memilih model mana yang ingin digunakan untuk prediksi (Random Forest, Logistic Regression, atau XGBoost) pada halaman

Model Demo, sehingga pengguna dapat membandingkan hasil prediksi antar model secara langsung di dalam aplikasi.

- Mempertimbangkan kembali pemisahan arsitektur menjadi backend API dan frontend terpisah apabila aplikasi ini nantinya perlu melayani banyak pengguna secara bersamaan di lingkungan produksi sesungguhnya, sebagaimana sempat direncanakan pada tahap proposal.

DAFTAR PUSTAKA

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

Brown, I., & Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3), 3446-3453.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284.

Lessmann, S., Baesens, B., Seow, H. V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), 124-136.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765-4774.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Santos, M. S., Soares, J. P., Abreu, P. H., Araujo, H., & Santos, J. (2018). Cross-validation for imbalanced datasets: Avoiding overoptimistic and overfitting approaches. *IEEE Computational Intelligence Magazine*, 13(4), 59-76.